

Tag each contact with an annotation

Tag each contact with a list of actions (operations) that your app can perform (for example: video calls and messaging).

Then, associate the ID of a contact to an ID that your app uses internally to identify that user.

C#

```
ContactAnnotationStore annotationStore = await

ContactManager.RequestAnnotationStoreAsync(ContactAnnotationStoreAccessType.
AppAnnotationsReadWrite);

ContactAnnotationList annotationList;

IReadOnlyList<ContactAnnotationList> annotationLists = await
annotationStore.FindAnnotationListsAsync();
if (0 == annotationLists.Count)
    annotationList = await annotationStore.CreateAnnotationListAsync();
else
    annotationList = annotationLists[0];

ContactAnnotation annotation = new ContactAnnotation();
annotation.ContactId = contact.Id;
annotation.RemoteId = "user22";

annotation.SupportedOperations = ContactAnnotationOperations.Message |
    ContactAnnotationOperations.AudioCall |
    ContactAnnotationOperations.VideoCall |
    ContactAnnotationOperations.ContactProfile;

await annotationList.TrySaveAnnotationAsync(annotation);
```

Register for each operation

In your package manifest, register for each operation that you listed in your annotation.

Register by adding protocol handlers to the `Extensions` element of the manifest.

XML

```
<Extensions>
  <uap:Extension Category="windows.protocol">
    <uap:Protocol Name="ms-contact-profile">
      <uap:DisplayName>TestProfileApp</uap:DisplayName>
    </uap:Protocol>
  </uap:Extension>
```

```

<uap:Extension Category="windows.protocol">
  <uap:Protocol Name="ms-ipmessaging">
    <uap:DisplayName>TestMsgApp</uap:DisplayName>
  </uap:Protocol>
</uap:Extension>
<uap:Extension Category="windows.protocol">
  <uap:Protocol Name="ms-voip-video">
    <uap:DisplayName>TestVideoApp</uap:DisplayName>
  </uap:Protocol>
</uap:Extension>
<uap:Extension Category="windows.protocol">
  <uap:Protocol Name="ms-voip-call">
    <uap:DisplayName>TestCallApp</uap:DisplayName>
  </uap:Protocol>
</uap:Extension>
</Extensions>

```

You can also add these in the **Declarations** tab of the manifest designer in Visual Studio.

Application Visual Assets Capabilities **Declarations** Content URIs Packaging

Use this page to add declarations and specify their properties.

Available Declarations:
 Select one... Add

Supported Declarations:

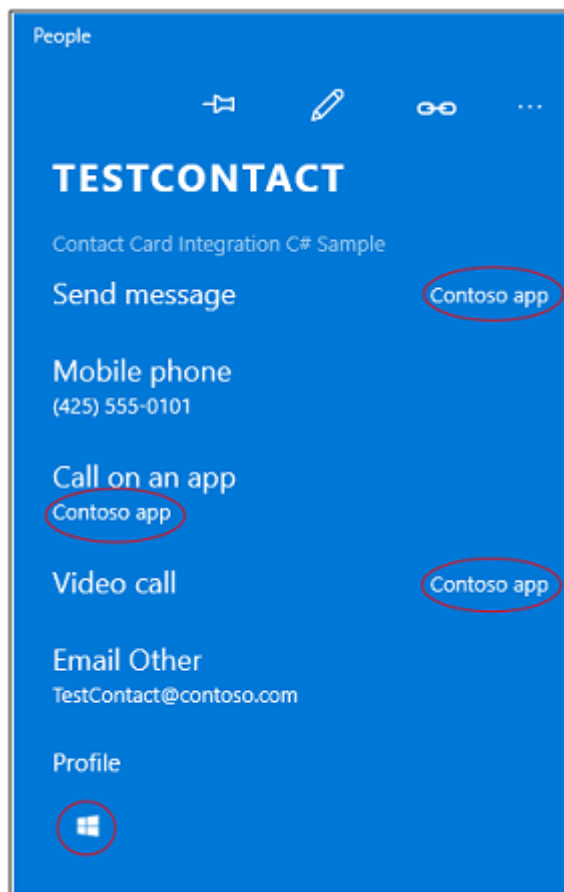
- Protocol
- Protocol
- Protocol
- Protocol Remove

Description:
 Register for URL Protocols, such as "mailto", on behalf of a Windows Store app.
 Multiple instances of this declaration are allowed in each app.
[More information](#)

Properties:
 Logo:
 Display name: TestCallApp
 Name: ms-voip-call
 App settings:

Find your app next to actions in a contact card

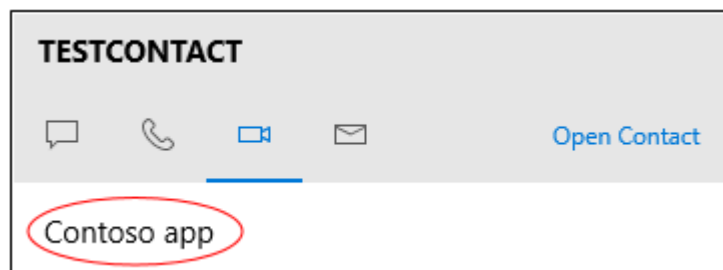
Open the People app. Your app appears next to each action (operation) that you specified in your annotation and package manifest.



If users choose your app for an action, it appears as the default app for that action the next time users open a contact card.

Find your app next to actions in a mini contact card

In mini contact cards, your app appears in tabs that represent actions.



Apps such as the Mail app open mini contact cards. Your app can open them too. This code shows you how to do that.

C#

```
public async void OpenContactCard(object sender, RoutedEventArgs e)
{
    // Get the selection rect of the button pressed to show contact card.
    FrameworkElement element = (FrameworkElement)sender;
```

```

        Windows.UI.Xaml.Media.GeneralTransform buttonTransform =
element.TransformToVisual(null);
        Windows.Foundation.Point point = buttonTransform.TransformPoint(new
Windows.Foundation.Point());
        Windows.Foundation.Rect rect =
            new Windows.Foundation.Rect(point, new
Windows.Foundation.Size(element.ActualWidth, element.ActualHeight));

        // helper method to find a contact just for illustrative purposes.
        Contact contact = await findContact("contoso@contoso.com");

        ContactManager.ShowContactCard(contact, rect,
Windows.UI.Popups.Placement.Default);
    }

```

To see more examples with mini contact cards, see [Contact cards sample](#).

Just like the contact card, each tab remembers the app that the user last used so it's easy for them to return to your app.

Perform operations when users select your app in a contact card

Override the [Application.OnActivated](#) method in your **App.cs** file and navigate users to a page in your app. The [Contact Card Integration Sample](#) shows one way to do that.

In the code behind file of the page, override the [Page.OnNavigatedTo](#) method. The contact card passes this method the name of operation and the ID of the user.

To start a video or audio call, see this sample: [VoIP sample](#). You'll find the complete API in the [Windows.ApplicationModel.Calls](#) namespace.

To facilitate messaging, see the [Windows.ApplicationModel.Chat](#) namespace.

You can also start another app. That's what this code does.

C#

```

protected override async void OnNavigatedTo(NavigationEventArgs e)
{
    base.OnNavigatedTo(e);

    var args = e.Parameter as ProtocolActivatedEventArgs;
    // Display the result of the protocol activation if we got here as a
    result of being activated for a protocol.

```

```
if (args != null)
{
    var options = new Windows.System.LauncherOptions();
    options.DisplayApplicationPicker = true;

    options.TargetApplicationPackageFamilyName = "ContosoApp";

    string launchString = args.uri.Scheme + ":" + args.uri.Query;
    var launchUri = new Uri(launchString);
    await Windows.System.Launcher.LaunchUriAsync(launchUri, options);
}
```

The `args.uri.scheme` property contains the name of the operation, and the `args.uri.Query` property contains the ID of the user.

Maps and location overview

Article • 06/21/2024

 Important

Bing Maps for Enterprise service retirement

The UWP [MapControl](#) and map services from the [Windows.Services.Maps](#) namespace rely on Bing Maps. Bing Maps for Enterprise is deprecated and will be retired, at which point the MapControl and services will no longer receive data.

For more information, see the [Bing Maps Developer Center](#)  and [Bing Maps documentation](#).

This section explains how you can display maps, use map services, find the location, and set up a geofence in your app. This section also shows you how to launch the Windows Maps app to a specific map, route, or a set of turn-by-turn directions.

[MapControl](#) and map services require a maps authentication key called a [MapServiceToken](#). For more info about getting and setting a maps authentication key, see [Request a maps authentication key](#).

Display maps

Display maps with 2D, 3D, or Streetside views in your app by using APIs from the [Windows.UI.Xaml.Controls.Maps](#) namespace. You can mark points of interest (POI) on the map by using pushpins, images, shapes, or XAML UI elements. You can also overlay tiled images or replace the map images altogether.

 **Expand table**

Topic	Description
Request a maps authentication key	Your app must be authenticated before it can use the MapControl and map services in the Windows.Services.Maps namespace. To authenticate your app, you must specify a maps authentication key. This article describes how to request a maps authentication key from the Bing Maps Developer Center  and add it to your app.
Display maps with 2D, 3D, and Streetside views	Display customizable maps in your app by using the MapControl class. This topic also introduces aerial 3D and Streetside views.

Topic	Description
Display points of interest (POI) on a map	Add points of interest (POI) to a map by using pushpins, images, shapes, and XAML UI elements.
Overlay tiled images on a map	Overlay third-party or custom tiled images on a map by using tile sources. Use tile sources to overlay specialized information such as weather data, population data, or seismic data; or use tile sources to replace the default map entirely.

Access map services

Add routes, directions, and geocoding capabilities to your app by using APIs from the [Windows.Services.Maps](#) namespace.

 Expand table

Topic	Description
Request a maps authentication key	Your app must be authenticated before it can use the MapControl and map services in the Windows.Services.Maps namespace. To authenticate your app, you must specify a maps authentication key. This article describes how to request a maps authentication key from the Bing Maps Developer Center  and add it to your app.
Display points of interest (POI) on a map	Add points of interest (POI) to a map by using pushpins, images, shapes, and XAML UI elements.
Display routes and directions	Request routes and directions, and display them in your app.
Perform geocoding and reverse geocoding	Convert addresses to geographic locations (geocoding) and convert geographic locations to addresses (reverse geocoding) by calling the methods of the MapLocationFinder class in the Windows.Services.Maps namespace.
Find and download map packages for offline use	In the past, your app had to direct users to the Settings app to download offline Maps. Now, you can use classes in the Windows.Services.Maps.OfflineMaps namespace to find downloaded packages in a given area (based on a Geopoint , GeoboundingBox , etc.). You can also check and listen for the downloaded status of map packages as well as start a download without requiring the user to leave your app. You'll find examples of how to do this in both the reference content and the Universal Windows Platform (UWP) map sample  .

Get the user's location

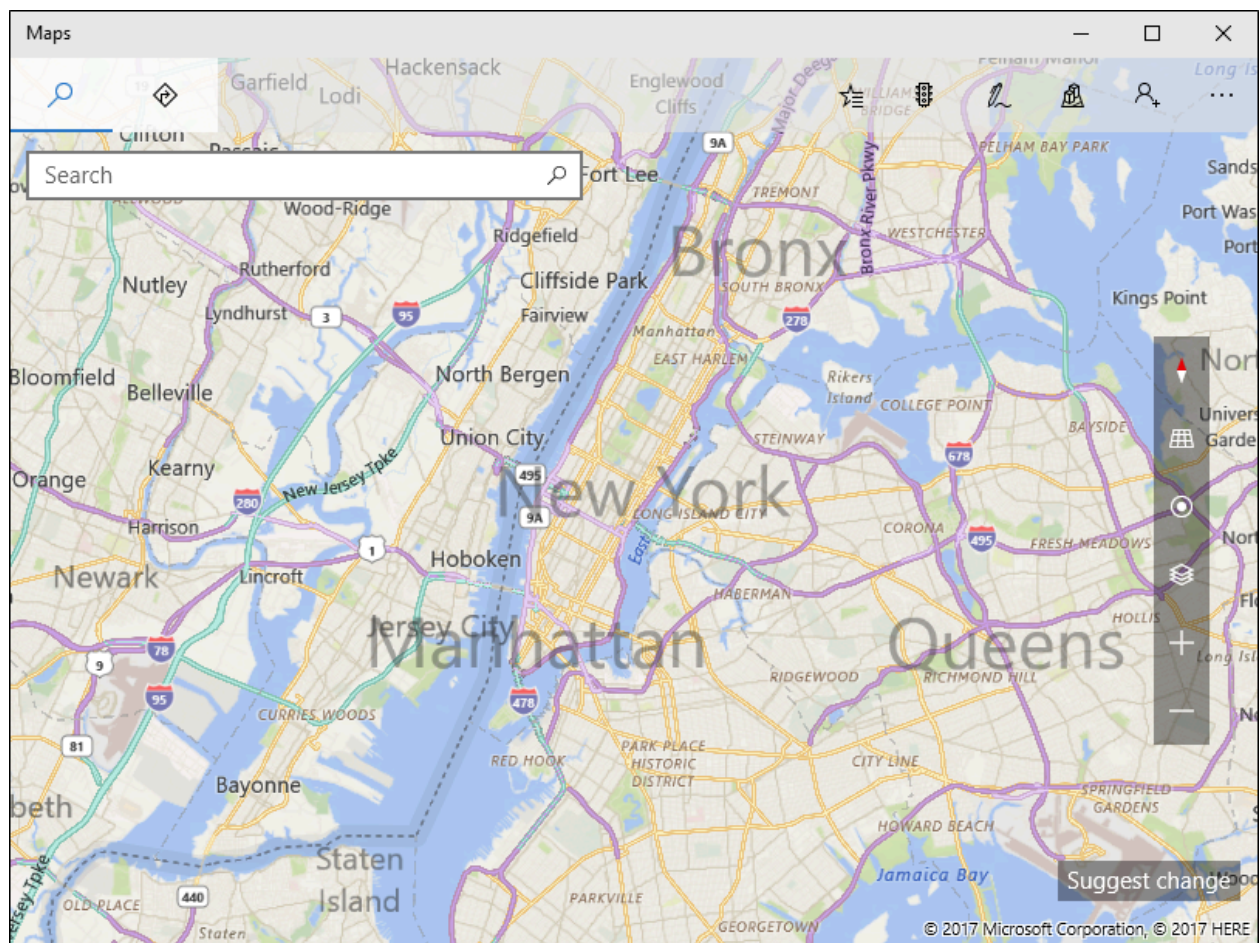
Get the user's current location and be notified when the location changes in your app by using APIs from the [Windows.Devices.Geolocation](#) namespace. These API members are also frequently used in parameters of the maps APIs. APIs from the [Windows.Devices.Geolocation.Geofencing](#) namespace notify your app when the user enters or exits a geofence (a predefined geographical area).

 Expand table

Topic	Description
Request a maps authentication key	Your app must be authenticated before it can use the MapControl and map services in the Windows.Services.Maps namespace. To authenticate your app, you must specify a maps authentication key. This article describes how to request a maps authentication key from the Bing Maps Developer Center  and add it to your app.
Design guidelines for location-aware apps	Performance guidelines for apps that require access to a user's location.
Get the user's location	Get access to the user's location, then retrieve it.
Guidelines for using Visits tracking	Learn how to use the powerful Visits Tracking feature for more practical location tracking.
Design guidance for geofencing	Performance guidelines for apps that utilize the geofencing feature.
Set up a geofence	Set up a geofence in your app, and learn how to handle notifications in the foreground and background.

Launch the Windows Maps app

Your app can launch the Windows Maps app as shown here to display specific maps and turn-by-turn directions. Rather than provide map functionality directly in your own app, consider using the Windows Maps app to provide that functionality. For more info, see [Launch the Windows Maps app](#).



Related topics

- [UWP map sample](#)
- [UWP geolocation sample](#)
- [Bing Maps Developer Center](#)
- [Get current location](#)
- [Design guidelines for location-aware apps](#)
- [Design guidelines for maps](#)
- [Design guidelines for privacy-aware apps](#)

Feedback

Was this page helpful?

[Yes](#)

[No](#)

[Provide product feedback](#) | [Get help at Microsoft Q&A](#)

Request a maps authentication key

Article • 06/21/2024

Important

Bing Maps for Enterprise service retirement

The UWP [MapControl](#) and map services from the [Windows.Services.Maps](#) namespace rely on Bing Maps. Bing Maps for Enterprise is deprecated and will be retired, at which point the MapControl and services will no longer receive data.

For more information, see the [Bing Maps Developer Center](#)  and [Bing Maps documentation](#).

Warning

Online maps services may not be available on older versions of Windows 10. On the following versions, MapControl may no longer display maps and APIs in the Windows.Services.Maps namespace may not return results:

- Windows 10, version 1607 and earlier versions: map services will be unavailable worldwide starting in October 2020
- Windows 10, version 1703 and earlier versions: map services are unavailable on [some devices sold in China](#)

Your app must be authenticated before it can use the [MapControl](#) and map services in the [Windows.Services.Maps](#) namespace. To authenticate your app, you must specify a maps authentication key. This topic describes how to request a maps authentication key from the [Bing Maps Developer Center](#)  and add it to your app.

Get a key

Create and manage map authentication keys for your Universal Windows apps using the [Bing Maps Developer Center](#) .

To create a new key

1. In your browser, navigate to the Bing Maps Developer Center (<https://www.bingmapsportal.com> .

2. If you are asked to sign in, enter your Microsoft account and click **Sign in**.
3. Choose the account to associate with your Bing Maps account. If you want to use your Microsoft account, click **Yes**. Otherwise, click **Sign in with another account**.
4. If you don't already have a Bing Maps account, create a new Bing Maps account. Enter the **Account Name**, **Contact Name**, **Company Name**, **Email Address**, and **Phone Number**. After accepting the terms of use, click **Create**.
5. Under the **My account** menu, click **My Keys**.
6. If you have previously created a key, click on the link to create a new key. Otherwise proceed to the Create Key form.
7. Complete the **Create Key** form and then click **Create**.
 - **Application name**: The name of your application.
 - **Application URL (optional)**: The URL of your application.
 - **Key type**: Select **Basic** or **Enterprise**.
 - **Application type**: Select **Windows Application** for use in your Universal Windows app.

This is an example of what the form looks like.

Create key

Application name *

Application URL

Key type * [What's This](#)

Application type *

[Create](#) [Cancel](#)

* Required field

To create Education, Broadcast or Not-for-Profit keys, please contact the Bing Maps account team at mpnet@microsoft.com.

- After you click **Create**, the new key appears below the **Create Key** form. Copy it to a safe place or immediately add it to your app, as described in the next step.

Add the key to your app

The map authentication key is required to use the [MapControl](#) and map services ([Windows.Services.Maps](#)) in your Universal Windows app. Add it to the map control and map service objects, as applicable.

To add the key to a map control

To authenticate the [MapControl](#), set the [MapServiceToken](#) property to the authentication key value. You can set this property in code or in XAML markup, depending on your preferences. For more info about using the **MapControl**, see [Display maps with 2D, 3D, and Streetside views](#).

- This example sets the **MapServiceToken** to the value of the authentication key in code.

CS

```
MapControl1.MapServiceToken = "abcdef-abcdefghijklmno";
```

- This example sets the **MapServiceToken** to the value of the authentication key in XAML markup.

XAML

```
<Maps:MapControl x:Name="MapControl1"  
    MapServiceToken="abcdef-abcdefghijklmno"/>
```

To add the key to map services

To use services in the [Windows.Services.Maps](#) namespace, set the **ServiceToken** property to the authentication key value. For more info about using map services, see [Display routes and directions](#) and [Perform geocoding and reverse geocoding](#).

- This example sets the **ServiceToken** to the value of the authentication key in code.

CS

```
MapService.ServiceToken = "abcdef-abcdefghijklmno";
```

Related topics

- [Bing Maps Developer Center](#) 
- [UWP map sample](#) 
- [Design guidelines for maps](#)
- [UWP traffic app sample](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Display maps with 2D, 3D, and Streetside views

Article • 06/21/2024

Important

Bing Maps for Enterprise service retirement

The UWP [MapControl](#) and map services from the [Windows.Services.Maps](#) namespace rely on Bing Maps. Bing Maps for Enterprise is deprecated and will be retired, at which point the MapControl and services will no longer receive data.

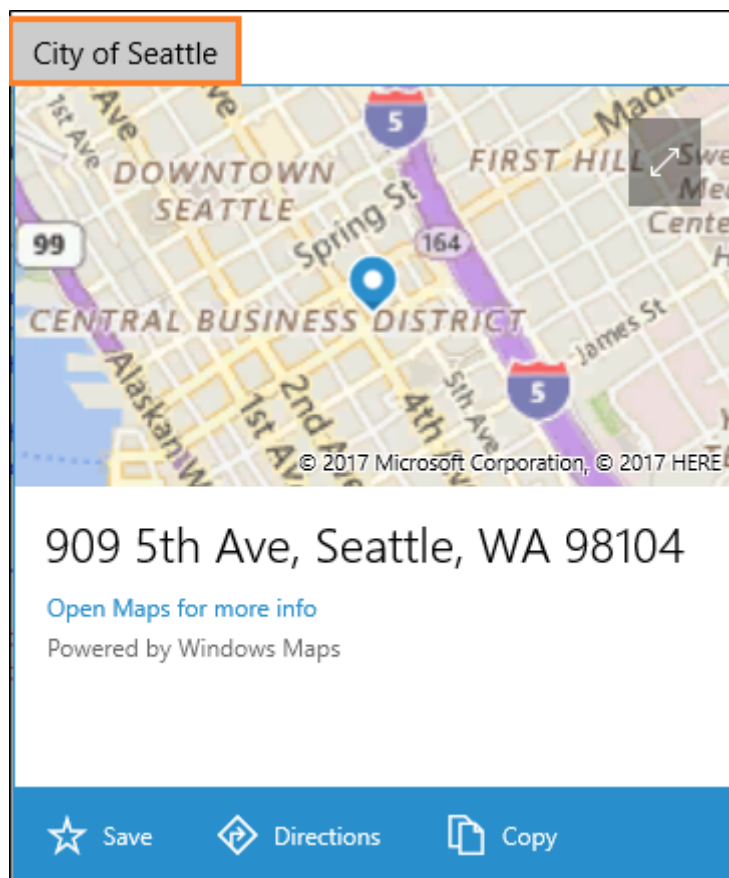
For more information, see the [Bing Maps Developer Center](#)  and [Bing Maps documentation](#). You can show a map in light dismissable window called a map *placecard* or in a full featured map control.

Download the [map sample](#)  to try out some the features described in this guide.

Display map in a placecard

You can show users a map inside of a light-weight pop-up window above, below or to the side of a UI element or an area of an app where the user touches. The map can show a city or address that relates to information in your app.

This placecard shows the city of Seattle.



Here's the code that makes Seattle appear in a placecard below a button.

C#

```
private void Seattle_Click(object sender, RoutedEventArgs e)
{
    Geopoint seattlePoint = new Geopoint
        (new BasicGeoposition { Latitude = 47.6062, Longitude = -122.3321
    });

    PlaceInfo spaceNeedlePlace = PlaceInfo.Create(seattlePoint);

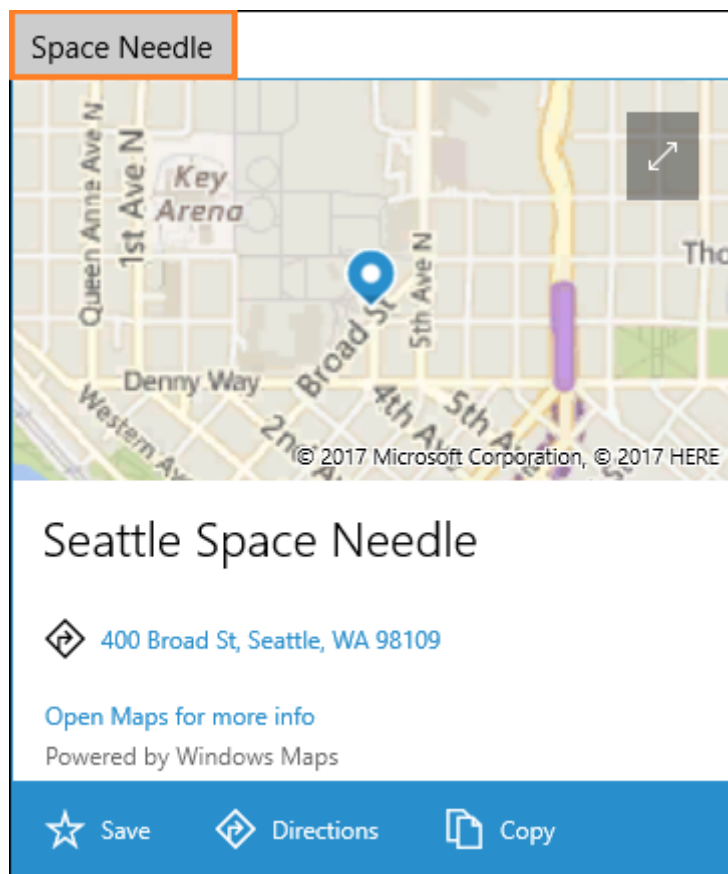
    FrameworkElement targetElement = (FrameworkElement)sender;

    GeneralTransform generalTransform =
targetElement.TransformToVisual((FrameworkElement)targetElement.Parent);

    Rect rectangle = generalTransform.TransformBounds(new Rect(new Point
        (targetElement.Margin.Left, targetElement.Margin.Top),
targetElement.RenderSize));

    spaceNeedlePlace.Show(rectangle, Windows.UI.Popups.Placement.Below);
}
```

This placecard shows the location of the Space Needle in Seattle.



Here's the code that makes the Space Needle appear in a placecard below a button.

C#

```
private void SpaceNeedle_Click(object sender, RoutedEventArgs e)
{
    Geopoint spaceNeedlePoint = new Geopoint
        (new BasicGeoposition { Latitude = 47.6205, Longitude = -122.3493
    });

    PlaceInfoCreateOptions options = new PlaceInfoCreateOptions();

    options.DisplayAddress = "400 Broad St, Seattle, WA 98109";
    options.DisplayName = "Seattle Space Needle";

    PlaceInfo spaceNeedlePlace = PlaceInfo.Create(spaceNeedlePoint,
options);

    FrameworkElement targetElement = (FrameworkElement)sender;

    GeneralTransform generalTransform =

targetElement.TransformToVisual((FrameworkElement)targetElement.Parent);

    Rect rectangle = generalTransform.TransformBounds(new Rect(new Point
        (targetElement.Margin.Left, targetElement.Margin.Top),
targetElement.RenderSize));
```

```
spaceNeedlePlace.Show(rectangle, Windows.UI.Popups.Placement.Below);  
}
```

Display map in a control

Use a map control to show rich and customizable map data in your app. A map control can display road maps, aerial, 3D, views, directions, search results, and traffic. On a map, you can display the user's location, directions, and points of interest. A map can also show aerial 3D views, Streetside views, traffic, transit, and local businesses.

Use a map control when you want a map within your app that allows users to view app-specific or general geographic information. Having a map control in your app means that users don't have to go outside your app to get that information.

ⓘ Note

If you don't mind users going outside your app, consider using the Windows Maps app to provide that information. Your app can launch the Windows Maps app to display specific maps, directions, and search results. For more info, see [Launch the Windows Maps app](#).

Add a map control to your app

Display a map on a XAML page by adding a [MapControl](#). To use the **MapControl**, you must declare the [Windows.UI.Xaml.Controls.Maps](#) namespace in the XAML page or in your code. If you drag the control from the Toolbox, this namespace declaration is added automatically. If you add the **MapControl** to the XAML page manually, you must add the namespace declaration manually at the top of the page.

The following example displays a basic map control and configures the map to display the zoom and tilt controls in addition to accepting touch inputs.

XAML

```
<Page  
    x:Class="MapsAndLocation1.DisplayMaps"  
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"  
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"  
    xmlns:local="using:MapsAndLocation1"  
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"  
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"  
    xmlns:Maps="using:Windows.UI.Xaml.Controls.Maps"  
    mc:Ignorable="d">
```

```

<Grid x:Name="pageGrid" Background="{ThemeResource
ApplicationPageBackgroundThemeBrush}">

    <Maps:MapControl
        x:Name="MapControl1"
        ZoomInteractionMode="GestureAndControl"
        TiltInteractionMode="GestureAndControl"
        MapServiceToken="EnterYourAuthenticationKeyHere"/>

</Grid>
</Page>

```

If you add the map control in your code, you must declare the namespace manually at the top of the code file.

```

C#

using Windows.UI.Xaml.Controls.Maps;
...

// Add the MapControl and the specify maps authentication key.
MapControl MapControl2 = new MapControl();
MapControl2.ZoomInteractionMode = MapInteractionMode.GestureAndControl;
MapControl2.TiltInteractionMode = MapInteractionMode.GestureAndControl;
MapControl2.MapServiceToken = "EnterYourAuthenticationKeyHere";
pageGrid.Children.Add(MapControl2);

```

Get and set a maps authentication key

Before you can use [MapControl](#) and map services, you must specify the maps authentication key as the value of the [MapServiceToken](#) property. In the previous examples, replace `EnterYourAuthenticationKeyHere` with the key you get from the [Bing Maps Developer Center](#). The text **Warning: MapServiceToken not specified** continues to appear below the control until you specify the maps authentication key. For more info about getting and setting a maps authentication key, see [Request a maps authentication key](#).

Set the location of a map

Point the map to any location that you want or use the user's current location.

Set a starting location for the map

Set the location to display on the map by specifying the [Center](#) property of the [MapControl](#) in your code or by binding the property in your XAML markup. The following example displays a map with the city of Seattle as its center.

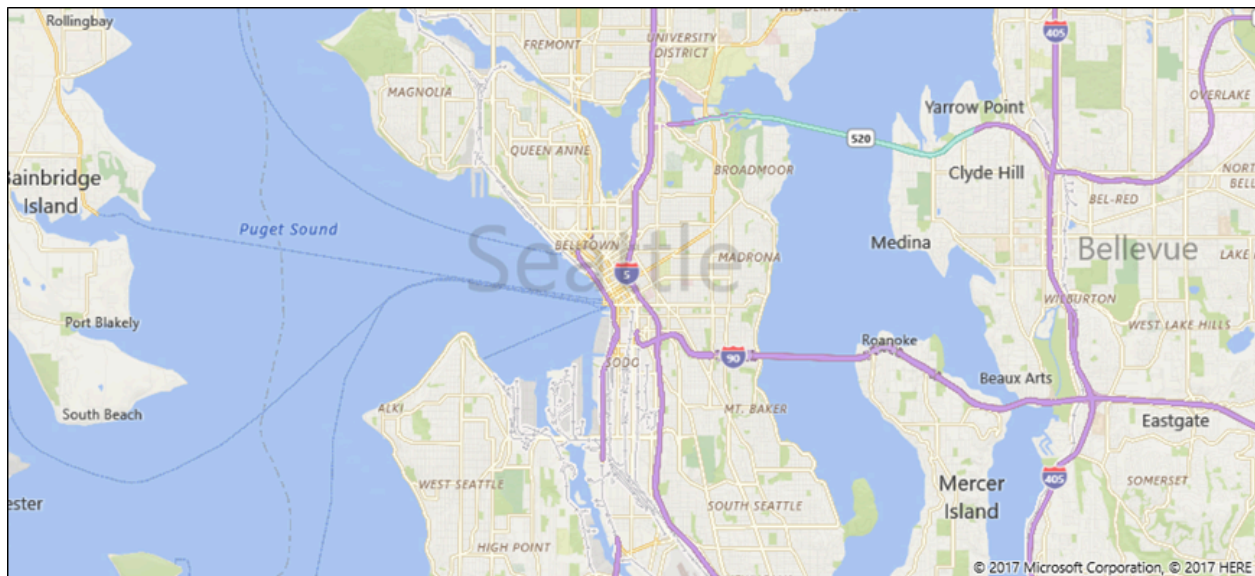
ⓘ Note

Since a string can't be converted to a [Geopoint](#), you can't specify a value for the [Center](#) property in XAML markup unless you use data binding. (This limitation also applies to the [MapControl.Location](#) attached property.)

C#

```
protected override void OnNavigatedTo(NavigationEventArgs e)
{
    // Specify a known location.
    BasicGeoposition cityPosition = new BasicGeoposition() { Latitude =
47.604, Longitude = -122.329 };
    Geopoint cityCenter = new Geopoint(cityPosition);

    // Set the map location.
    MapControl1.Center = cityCenter;
    MapControl1.ZoomLevel = 12;
    MapControl1.LandmarksVisible = true;
}
```



Set the current location of the map

Before your app can access the user's location, your app must call the [RequestAccessAsync](#) method. At that time, your app must be in the foreground and [RequestAccessAsync](#) must be called from the UI thread. Until the user grants your app permission to their location, your app can't access location data.

Get the current location of the device (if location is available) by using the [GetGeopositionAsync](#) method of the [Geolocator](#) class. To obtain the corresponding [Geopoint](#), use the [Point](#) property of the geoposition's geocoordinate. For more info, see [Get current location](#).

C#

```
// Set your current location.
var accessStatus = await Geolocator.RequestAccessAsync();
switch (accessStatus)
{
    case GeolocationAccessStatus.Allowed:

        // Get the current location.
        Geolocator geolocator = new Geolocator();
        Geoposition pos = await geolocator.GetGeopositionAsync();
        Geopoint myLocation = pos.Coordinate.Point;

        // Set the map location.
        MapControl1.Center = myLocation;
        MapControl1.ZoomLevel = 12;
        MapControl1.LandmarksVisible = true;
        break;

    case GeolocationAccessStatus.Denied:
        // Handle the case if access to location is denied.
        break;

    case GeolocationAccessStatus.Unspecified:
        // Handle the case if an unspecified error occurs.
        break;
}
```

When you display your device's location on a map, consider displaying graphics and setting the zoom level based on the accuracy of the location data. For more info, see [Guidelines for location-aware apps](#).

Change the location of the map

To change the location that appears in a 2D map, call one of the overloads of the [TrySetViewAsync](#) method. Use that method to specify new values for [Center](#), [ZoomLevel](#), [Heading](#), and [Pitch](#). You can also specify an optional animation to use when the view changes by providing a constant from the [MapAnimationKind](#) enumeration.

To change the location of a 3D map, use the [TrySetSceneAsync](#) method instead. For more info, see [Display aerial 3D views](#).

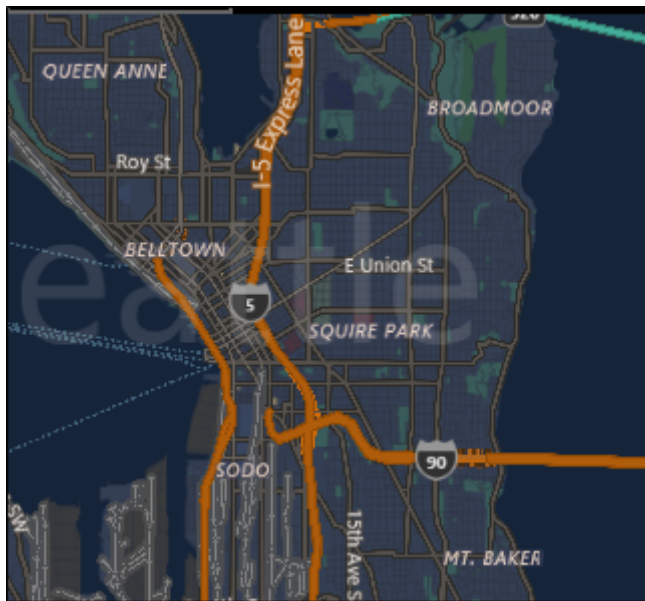
Call the [TrySetViewBoundsAsync](#) method to display the contents of a [GeoboundingBox](#) on the map. Use this method, for example, to display a route or a portion of a route on the map. For more info, see [Display routes and directions on a map](#).

Change the appearance of a map

To customize the look and feel of the map, set the [StyleSheet](#) property of the map control to any of the existing [MapStyleSheet](#) objects.

C#

```
myMap.StyleSheet = MapStyleSheet.RoadDark();
```



You can also use JSON to define custom styles and then use that JSON to create a [MapStyleSheet](#) object.

Style sheet JSON can be created interactively using the [Map Style Sheet Editor](#) application.

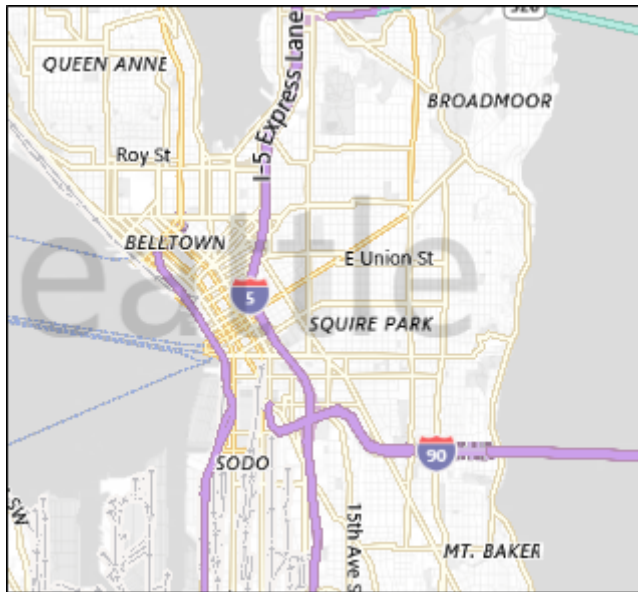
C#

```
myMap.StyleSheet = MapStyleSheet.ParseFromJson(@"  
  {  
    ""version"": ""1.0"",  
    ""settings"": {  
      ""landColor"": ""#FFFFFF"",  
      ""spaceColor"": ""#000000""  
    },  
    ""elements"": {  
      ""mapElement"": {  
        ""labelColor"": ""#000000"",  
        ""labelOutlineColor"": ""#FFFFFF""  
      }  
    }  
  }  
");
```

```

    },
    "water": {
        "fillColor": "#DDDDDD"
    },
    "area": {
        "fillColor": "#EEEEEE"
    },
    "political": {
        "borderStrokeColor": "#CCCCCC",
        "borderOutlineColor": "#00000000"
    }
}
});

```



For the complete JSON entry reference, see [Map style sheet reference](#).

You can start with an existing sheet and then use JSON to override any elements that you want. This example, starts with an existing style and uses JSON to change only the color of water areas.

C#

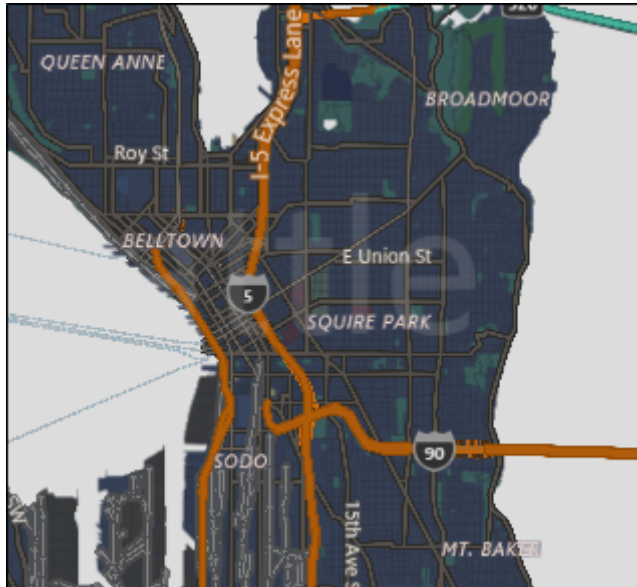
```

MapStyleSheet \customSheet = MapStyleSheet.ParseFromJson(@"
{
    "version": "1.0",
    "elements": {
        "water": {
            "fillColor": "#DDDDDD"
        }
    }
}
");

MapStyleSheet builtInSheet = MapStyleSheet.RoadDark();

```

```
myMap.StyleSheet = MapStyleSheet.Combine(new List<MapStyleSheet> {  
    builtInSheet, customSheet });
```



ⓘ Note

Styles that you define in the second style sheet override the styles in the first.

Set orientation and perspective

Zoom in, zoom out, rotate, and tilt the map's camera to get just the right angle for the effect that you want. Try these properties.

- Set the **center** of the map to a geographic point by setting the [Center](#) property.
- Set the **zoom level** of the map by setting the [ZoomLevel](#) property to a value between 1 and 20.
- Set the **rotation** of the map by setting the [Heading](#) property, where 0 or 360 degrees = North, 90 = East, 180 = South, and 270 = West.
- Set the **tilt** of the map by setting the [DesiredPitch](#) property to a value between 0 and 65 degrees.

Show and hide map features

Show or hide map features such as roads and landmarks by setting the values of the following properties of the [MapControl](#).

- Display **buildings and landmarks** on the map by enabling or disabling the [LandmarksVisible](#) property.

ⓘ Note

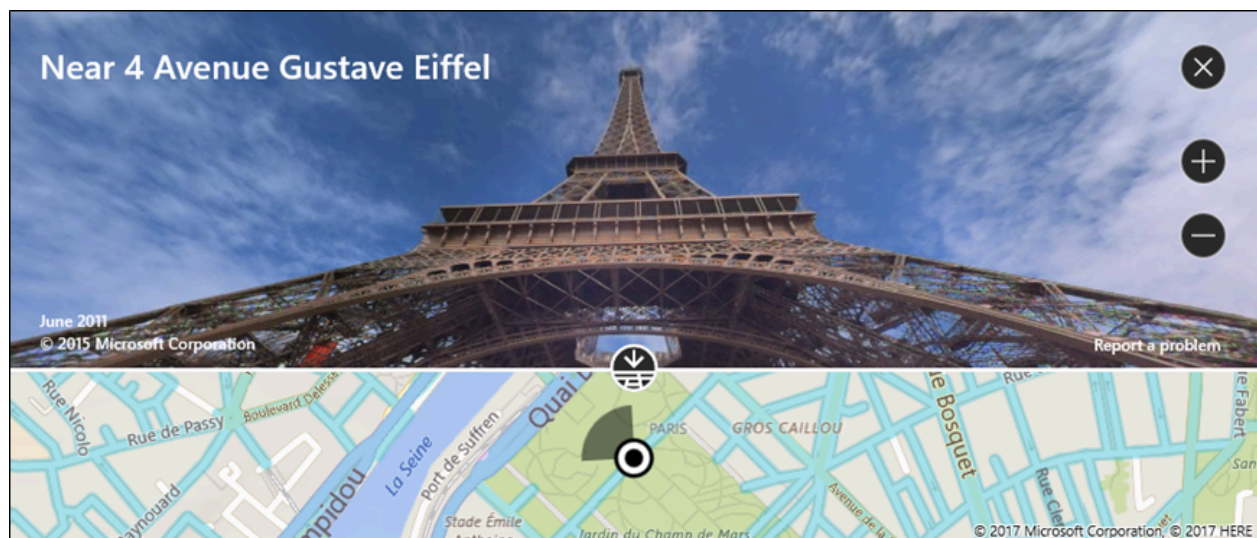
You can show or hide buildings, but you can't prevent them from appearing 3 dimensions.

- Display **pedestrian features** such as public stairs on the map by enabling or disabling the [PedestrianFeaturesVisible](#) property.
- Display **traffic** on the map by enabling or disabling the [TrafficFlowVisible](#) property.
- Specify whether the **watermark** is displayed on the map by setting the [WatermarkMode](#) property to one of the [MapWatermarkMode](#) constants.
- Display a **driving or walking route** on the map by adding a [MapRouteView](#) to the [Routes](#) collection of the Map control. For more info and an example, see [Display routes and directions on a map](#).

For info about how to display pushpins, shapes, and XAML controls in the [MapControl](#), see [Display points of interest \(POI\) on a map](#).

Display Streetside views

A Streetside view is a street-level perspective of a location that appears on top of the map control.



Consider the experience "inside" the Streetside view separate from the map originally displayed in the map control. For example, changing the location in the Streetside view does not change the location or appearance of the map "under" the Streetside view. After you close the Streetside view (by clicking the X in the upper right corner of the control), the original map remains unchanged.

To display a Streetside view

1. Determine if Streetside views are supported on the device by checking [IsStreetsideSupported](#).
2. If Streetside view is supported, create a [StreetsidePanorama](#) near the specified location by calling [FindNearbyAsync](#).
3. Determine if a nearby panorama was found by checking if the [StreetsidePanorama](#) is not null
4. If a nearby panorama was found, create a [StreetsideExperience](#) for the map control's [CustomExperience](#) property.

This example shows how to display a Streetside view similar to the previous image.

ⓘ Note

The overview map will not appear if the map control is sized too small.

C#

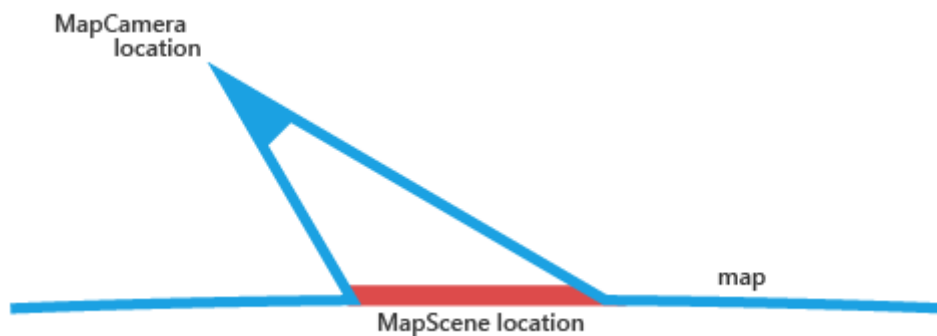
```
private async void showStreetsideView()
{
    // Check if Streetside is supported.
    if (MapControl1.IsStreetsideSupported)
    {
        // Find a panorama near Avenue Gustave Eiffel.
        BasicGeoposition cityPosition = new BasicGeoposition() { Latitude = 48.858, Longitude = 2.295 };
        Geopoint cityCenter = new Geopoint(cityPosition);
        StreetsidePanorama panoramaNearCity = await StreetsidePanorama.FindNearbyAsync(cityCenter);

        // Set the Streetside view if a panorama exists.
        if (panoramaNearCity != null)
        {
            // Create the Streetside view.
            StreetsideExperience ssView = new StreetsideExperience(panoramaNearCity);
            ssView.OverviewMapVisible = true;
            MapControl1.CustomExperience = ssView;
        }
    }
    else
    {
        // If Streetside is not supported
        ContentDialog viewNotSupportedDialog = new ContentDialog()
        {
            Title = "Streetside is not supported",
            Content = "\nStreetside views are not supported on this device.",
            PrimaryButtonText = "OK"
        }
    }
}
```

```
};  
await viewNotSupportedDialog.ShowAsync();  
}  
}
```

Display aerial 3D views

Specify a 3D perspective of the map by using the [MapScene](#) class. The map scene represents the 3D view that appears in the map. The [MapCamera](#) class represents the position of the camera that would display such a view.



To make buildings and other features on the map surface appear in 3D, set the map control's [Style](#) property to [MapStyle.Aerial3DWithRoads](#). This is an example of a 3D view with the [Aerial3DWithRoads](#) style.



To display a 3D view

1. Determine if 3D views are supported on the device by checking [Is3DSupported](#).
2. If 3D views is supported, set the map control's [Style](#) property to [MapStyle.Aerial3DWithRoads](#).
3. Create a [MapScene](#) object using one of the many [CreateFrom](#) methods, such as [CreateFromLocationAndRadius](#) and [CreateFromCamera](#).

4. Call [TrySetSceneAsync](#) to display the 3D view. You can also specify an optional animation to use when the view changes by providing a constant from the [MapAnimationKind](#) enumeration.

This example shows how to display a 3D view.

C#

```
private async void display3DLocation()
{
    if (MapControl1.Is3DSupported)
    {
        // Set the aerial 3D view.
        MapControl1.Style = MapStyle.Aerial3DWithRoads;

        // Specify the location.
        BasicGeoposition hwGeoposition = new BasicGeoposition() { Latitude =
43.773251, Longitude = 11.255474};
        Geopoint hwPoint = new Geopoint(hwGeoposition);

        // Create the map scene.
        MapScene hwScene = MapScene.CreateFromLocationAndRadius(hwPoint,

80, /* show this many meters around */

0, /* looking at it to the North*/

60 /* degrees pitch */);
        // Set the 3D view with animation.
        await MapControl1.TrySetSceneAsync(hwScene, MapAnimationKind.Bow);
    }
    else
    {
        // If 3D views are not supported, display dialog.
        ContentDialog viewNotSupportedDialog = new ContentDialog()
        {
            Title = "3D is not supported",
            Content = "\n3D views are not supported on this device.",
            PrimaryButtonText = "OK"
        };
        await viewNotSupportedDialog.ShowAsync();
    }
}
```

Get info about locations

Get info about locations on the map by calling the following methods of the [MapControl](#).

- [TryGetLocationFromOffset](#) method - Get the geographic location that corresponds to the specified point in the viewport of the Map control.
- [GetOffsetFromLocation](#) method - Get the point in the viewport of the Map control that corresponds to the specified geographic location.
- [IsLocationInView](#) method - Determine whether the specified geographic location is currently visible in the viewport of the Map control.
- [FindMapElementsAtOffset](#) method - Get the elements on the map located at the specified point in the viewport of the Map control.

Handle interaction and changes

Handle user input gestures on the map by handling the following events of the [MapControl](#). Get info about the geographic location on the map and the physical position in the viewport where the gesture occurred by checking the values of the [Location](#) and [Position](#) properties of the [MapInputEventArgs](#).

- [MapTapped](#)
- [MapDoubleTapped](#)
- [MapHolding](#)

Determine whether the map is loading or completely loaded by handling the control's [LoadingStatusChanged](#) event.

Handle changes that happen when the user or the app changes the settings of the map by handling the following events of the [MapControl](#).

- [CenterChanged](#)
- [HeadingChanged](#)
- [PitchChanged](#)
- [ZoomLevelChanged](#)

Best practice recommendations

- Use ample screen space (or the entire screen) to display the map so that users don't have to pan and zoom excessively to view geographical information.
- If the map is only used to present a static, informational view, then using a smaller map might be more appropriate. If you go with a smaller, static map, base its dimensions on usability—small enough to conserve enough screen real estate, but large enough to remain legible.

- Embed the points of interest in the map scene using [map elements](#); any additional information can be displayed as transient UI that overlays the map scene.

Related topics

- [Bing Maps Developer Center](#) 
- [UWP map sample](#) 
- [Get current location](#)
- [Design guidelines for location-aware apps](#)
- [MapControl](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Display points of interest on a map

Article • 06/21/2024

❗ Important

Bing Maps for Enterprise service retirement

The UWP [MapControl](#) and map services from the [Windows.Services.Maps](#) namespace rely on Bing Maps. Bing Maps for Enterprise is deprecated and will be retired, at which point the MapControl and services will no longer receive data.

For more information, see the [Bing Maps Developer Center](#) [↗] and [Bing Maps documentation](#).

❗ Note

[MapControl](#) and map services require a maps authentication key called a [MapServiceToken](#). For more info about getting and setting a maps authentication key, see [Request a maps authentication key](#).

Add points of interest (POI) to a map using pushpins, images, shapes, and XAML UI elements. A POI is a specific point on the map that represents something of interest. For example, the location of a business, city, or friend.

Display pushpins, images, and shapes on the map by adding [MapIcon](#), [MapBillboard](#), [MapPolygon](#), and [MapPolyline](#) objects to a [MapElements](#) collection of a [MapElementsLayer](#) object. Then, add that layer object to the [Layers](#) collection of a map control.

❗ Note

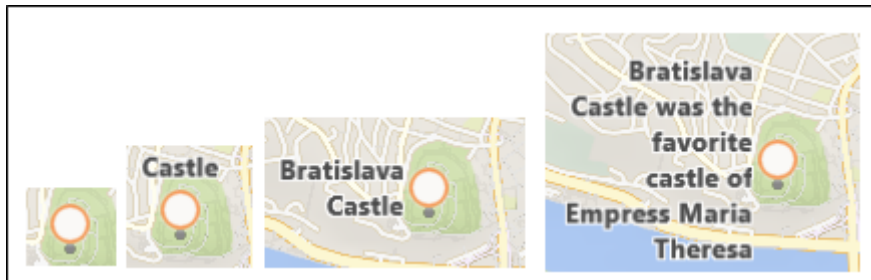
In previous releases, this guide showed you how to add map elements to the [MapElements](#) collection. While you can still use this approach, you'll miss out on some of the advantages of the new map layer model. To learn more, see the [Working with layers](#) section of this guide.

You can also display XAML user interface elements such as a [Button](#), a [HyperlinkButton](#), or a [TextBlock](#) on the map by adding them to the [MapItemsControl](#) or as [Children](#) of the [MapControl](#).

If you have a large number of elements to place on the map, consider [overlying tiled images on the map](#). To display roads on the map, see [Display routes and directions](#)

Add a pushpin

Display an image such a pushpin, with optional text, on the map by using the **MapIcon** class. You can accept the default image or provide a custom image by using the **Image** property. The following image displays the default image for a **MapIcon** with no value specified for the **Title** property, with a short title, with a long title, and with a very long title.



The following example shows a map of the city of Seattle and adds a **MapIcon** with the default image and an optional title to indicate the location of the Space Needle. It also centers the map over the icon and zooms in. For general info about using the map control, see [Display maps with 2D, 3D, and Streetside views](#).

C#

```
public void AddSpaceNeedleIcon()
{
    var MyLandmarks = new List<MapElement>();

    BasicGeoposition snPosition = new BasicGeoposition { Latitude = 47.620,
    Longitude = -122.349 };
    Geopoint snPoint = new Geopoint(snPosition);

    var spaceNeedleIcon = new MapIcon
    {
        Location = snPoint,
        NormalizedAnchorPoint = new Point(0.5, 1.0),
        ZIndex = 0,
        Title = "Space Needle"
    };

    MyLandmarks.Add(spaceNeedleIcon);

    var LandmarksLayer = new MapElementsLayer
    {
        ZIndex = 1,
        MapElements = MyLandmarks
    };
}
```

```

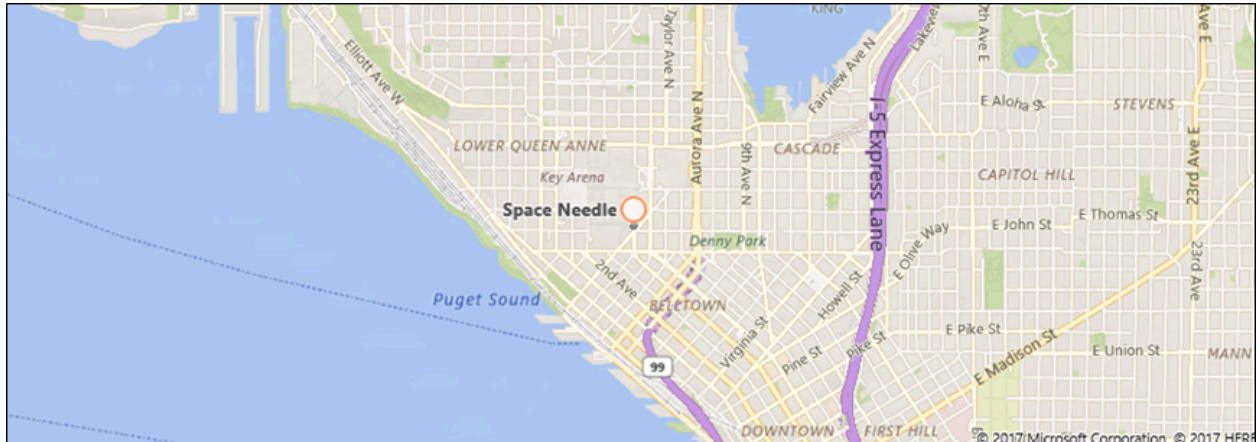
myMap.Layers.Add(LandmarksLayer);

myMap.Center = snPoint;
myMap.ZoomLevel = 14;

}

```

This example displays the following POI on the map (the default image in the center).



The following line of code displays the **MapIcon** with a custom image saved in the Assets folder of the project. The **Image** property of the **MapIcon** expects a value of type **RandomAccessStreamReference**. This type requires a **using** statement for the **Windows.Storage.Streams** namespace.

! Note

If you use the same image for multiple map icons, declare the **RandomAccessStreamReference** at the page or app level for the best performance.

C#

```

MapIcon1.Image =
    RandomAccessStreamReference.CreateFromUri(new Uri("ms-
    appx:///Assets/customicon.png"));

```

Keep these considerations in mind when working with the **MapIcon** class:

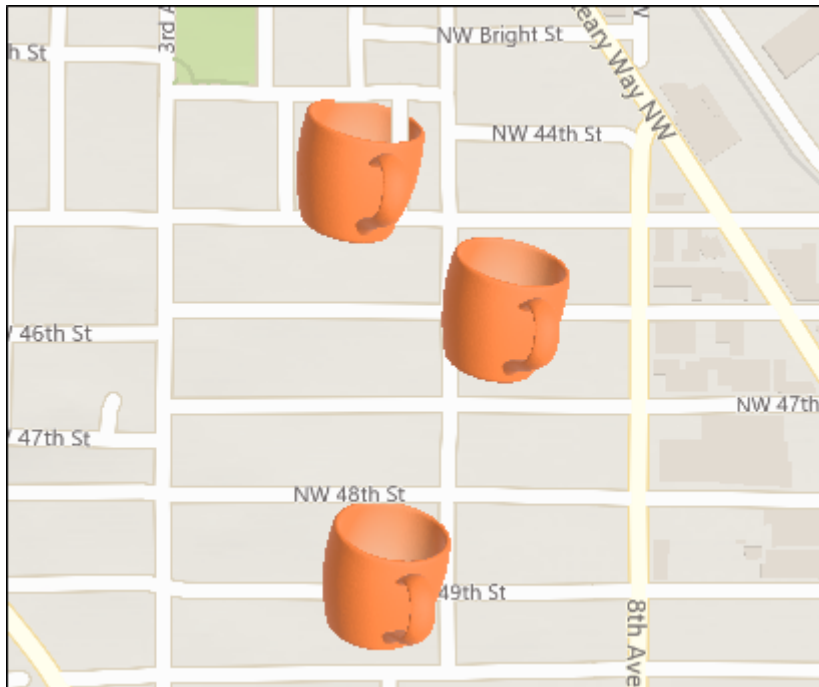
- The **Image** property supports a maximum image size of 2048×2048 pixels.
- By default, the map icon's image is not guaranteed to be shown. It may be hidden when it obscures other elements or labels on the map. To keep it visible, set the map icon's **CollisionBehaviorDesired** property to **MapElementCollisionBehavior.RemainVisible**.

- The optional **Title** of the **MapIcon** is not guaranteed to be shown. If you don't see the text, zoom out by decreasing the value of the **ZoomLevel** property of the **MapControl**.
- When you display a **MapIcon** image that points to a specific location on the map - for example, a pushpin or an arrow - consider setting the value of the **NormalizedAnchorPoint** property to the approximate location of the pointer on the image. If you leave the value of **NormalizedAnchorPoint** at its default value of (0, 0), which represents the upper left corner of the image, changes in the **ZoomLevel** of the map may leave the image pointing to a different location.
- If you don't explicitly set an **Altitude** and **AltitudeReferenceSystem**, the **MapIcon** will be placed on the surface.

Add a 3D pushpin

You can add three-dimensional objects to a map. Use the **MapModel3D** class to import a 3D object from a **3D Manufacturing Format (3MF)** [file](#).

This image uses 3D coffee cups to mark the locations of coffee shops in a neighborhood.



The following code adds a coffee cup to the map by using importing a 3MF file. To keep things simple, this code adds the image to the center of the map, but your code would likely add the image to a specific location.

C#

```
public async void Add3DMapModel()
{
```

```

var mugStreamReference = RandomAccessStreamReference.CreateFromUri
    (new Uri("ms-appx:///Assets/mug.3mf"));

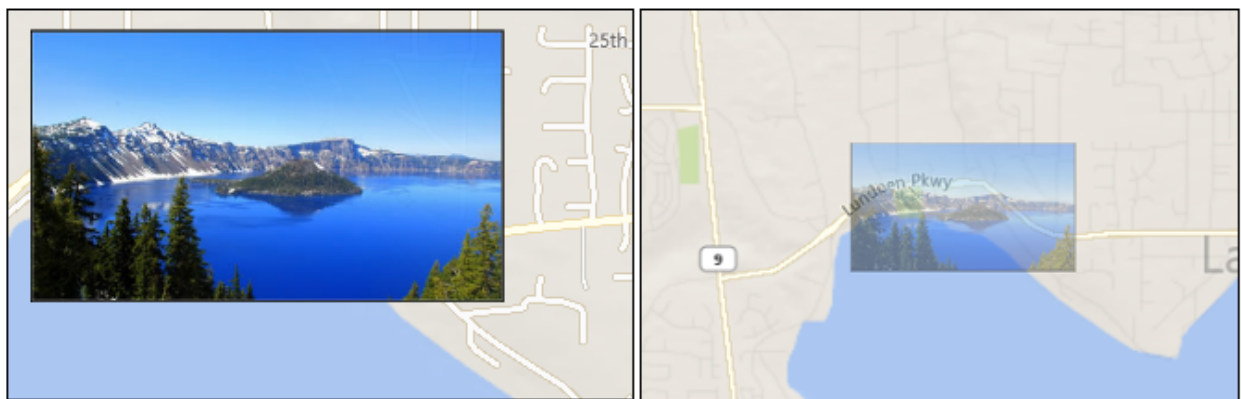
var myModel = await MapModel3D.CreateFrom3MFAsync(mugStreamReference,
    MapModel3DShadingOption.Smooth);

myMap.Layers.Add(new MapElementsLayer
{
    ZIndex = 1,
    MapElements = new List<MapElement>
    {
        new MapElement3D
        {
            Location = myMap.Center,
            Model = myModel,
        },
    },
});
}

```

Add an image

Display large images that relate to map locations such as a picture of a restaurant or a landmark. As users zoom out, the image will shrink proportionally in size to enable the user to view more of the map. This is a bit different than a [MapIcon](#) which marks a specific location, is typically small, and remains the same size as users zoom in and out of a map.



The following code shows the [MapBillboard](#) presented in the image above.

C#

```

public void AddLandmarkPhoto()
{
    // Create MapBillboard.

    RandomAccessStreamReference mapBillboardStreamReference =
        RandomAccessStreamReference.CreateFromUri(new Uri("ms-

```

```

appx:///Assets/billboard.jpg"));

    var mapBillboard = new MapBillboard(myMap.ActualCamera)
    {
        Location = myMap.Center,
        NormalizedAnchorPoint = new Point(0.5, 1.0),
        Image = mapBillboardStreamReference
    };

    // Add MapBillboard to a layer on the map control.

    var MyLandmarkPhotos = new List<MapElement>();

    MyLandmarkPhotos.Add(mapBillboard);

    var LandmarksPhotoLayer = new MapElementsLayer
    {
        ZIndex = 1,
        MapElements = MyLandmarkPhotos
    };

    myMap.Layers.Add(LandmarksPhotoLayer);
}

```

There's three parts of this code worth examining a little closer: The image, the reference camera, and the [NormalizedAnchorPoint](#) property.

Image

This example shows a custom image saved in the **Assets** folder of the project. The [Image](#) property of the [MapBillboard](#) expects a value of type [RandomAccessStreamReference](#). This type requires a **using** statement for the [Windows.Storage.Streams](#) namespace.

⚠ Note

If you use the same image for multiple map icons, declare the [RandomAccessStreamReference](#) at the page or app level for the best performance.

Reference camera

Because a [MapBillboard](#) image scales in and out as the [ZoomLevel](#) of the map changes, it's important to define where in that [ZoomLevel](#) the image appears at a normal 1x scale. That position is defined in the reference camera of the [MapBillboard](#), and to set it, you'll have to pass a [MapCamera](#) object into the constructor of the [MapBillboard](#).

You can define the position that you want in a [Geopoint](#), and then use that [Geopoint](#) to create a [MapCamera](#) object. However, in this example, we're just using the [MapCamera](#) object returned by the [ActualCamera](#) property of the map control. This is the map's internal camera. The current position of that camera becomes the reference camera position; the position where the [MapBillboard](#) image appears at 1x scale.

If your app gives users the ability to zoom out on the map, the image decreases in size because the map's internal camera is rising in altitude while the image at 1x scale remains fixed at the reference camera's position.

NormalizedAnchorPoint

The [NormalizedAnchorPoint](#) is the point of the image that is anchored to the [Location](#) property of the [MapBillboard](#). The point 0.5,1 is the bottom center of the image. Because we've set the [Location](#) property of the [MapBillboard](#) to the center of the map's control, the bottom center of the image will be anchored at the center of the map's control. If you want your image to appear centered directly over a point, set the [NormalizedAnchorPoint](#) to 0.5,0.5.

Add a shape

Display a multi-point shape on the map by using the [MapPolygon](#) class. The following example, from the [UWP map sample](#) [↗](#), displays a red box with blue border on the map.

C#

```
public void HighlightArea()
{
    // Create MapPolygon.

    double centerLatitude = myMap.Center.Position.Latitude;
    double centerLongitude = myMap.Center.Position.Longitude;

    var mapPolygon = new MapPolygon
    {
        Path = new Geopath(new List<BasicGeoposition> {
            new BasicGeoposition() {Latitude=centerLatitude+0.0005,
Longitude=centerLongitude-0.001 },
            new BasicGeoposition() {Latitude=centerLatitude-0.0005,
Longitude=centerLongitude-0.001 },
            new BasicGeoposition() {Latitude=centerLatitude-0.0005,
Longitude=centerLongitude+0.001 },
            new BasicGeoposition() {Latitude=centerLatitude+0.0005,
Longitude=centerLongitude+0.001 },
        }),
        ZIndex = 1,
    }
```

```

        FillColor = Colors.Red,
        StrokeColor = Colors.Blue,
        StrokeThickness = 3,
        StrokeDashed = false,
    };

    // Add MapPolygon to a layer on the map control.
    var MyHighlights = new List<MapElement>();

    MyHighlights.Add(mapPolygon);

    var HighlightsLayer = new MapElementsLayer
    {
        ZIndex = 1,
        MapElements = MyHighlights
    };

    myMap.Layers.Add(HighlightsLayer);
}

```

Add a line

Display a line on the map by using the [MapPolyline](#) class. The following example, from the [UWP map sample](#), displays a dashed line on the map.

C#

```

public void DrawLineOnMap()
{
    // Create Polyline.

    double centerLatitude = myMap.Center.Position.Latitude;
    double centerLongitude = myMap.Center.Position.Longitude;
    var mapPolyline = new MapPolyline
    {
        Path = new Geopath(new List<BasicGeoposition> {
            new BasicGeoposition() {Latitude=centerLatitude-0.0005,
Longitude=centerLongitude-0.001 },
            new BasicGeoposition() {Latitude=centerLatitude+0.0005,
Longitude=centerLongitude+0.001 },
        }),
        StrokeColor = Colors.Black,
        StrokeThickness = 3,
        StrokeDashed = true,
    };

    // Add Polyline to a layer on the map control.

    var MyLines = new List<MapElement>();

    MyLines.Add(mapPolyline);
}

```

```

var LinesLayer = new MapElementsLayer
{
    ZIndex = 1,
    MapElements = MyLines
};

myMap.Layers.Add(LinesLayer);

}

```

Add XAML

Display custom UI elements on the map using XAML. Position XAML on the map by specifying the location and normalized anchor point of the XAML.

- Set the location on the map where the XAML is placed by calling [SetLocation](#).
- Set the relative location on the XAML that corresponds to the specified location by calling [SetNormalizedAnchorPoint](#).

The following example shows a map of the city of Seattle and adds a XAML [Border](#) control to indicate the location of the Space Needle. It also centers the map over the area and zooms in. For general info about using the map control, see [Display maps with 2D, 3D, and Streetside views](#).

C#

```

private void displayXAMLButton_Click(object sender, RoutedEventArgs e)
{
    // Specify a known location.
    BasicGeoposition snPosition = new BasicGeoposition { Latitude = 47.620,
    Longitude = -122.349 };
    Geopoint snPoint = new Geopoint(snPosition);

    // Create a XAML border.
    Border border = new Border
    {
        Height = 100,
        Width = 100,
        BorderBrush = new SolidColorBrush(Windows.UI.Colors.Blue),
        BorderThickness = new Thickness(5),
    };

    // Center the map over the POI.
    MapControl1.Center = snPoint;
    MapControl1.ZoomLevel = 14;

    // Add XAML to the map.
    MapControl1.Children.Add(border);
}

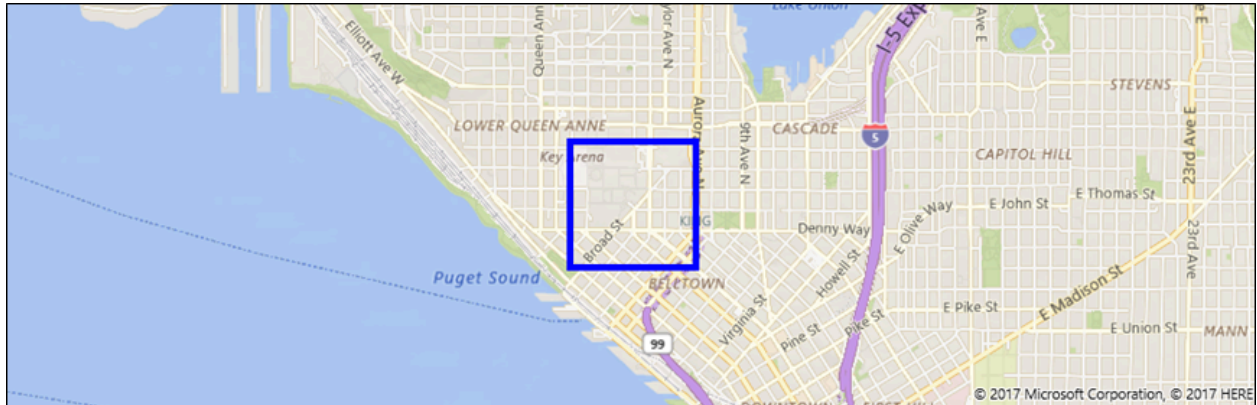
```

```

MapControl.SetLocation(border, snPoint);
MapControl.SetNormalizedAnchorPoint(border, new Point(0.5, 0.5));
}

```

This example displays a blue border on the map.



The next examples show how to add XAML UI elements directly in the XAML markup of the page using data binding. As with other XAML elements that display content, **Children** is the default content property of the **MapControl** and does not have to be specified explicitly in XAML markup.

This example shows how to display two XAML controls as implicit children of the **MapControl**. These controls appear on the map at the data bound locations.

XAML

```

<maps:MapControl>
    <TextBox Text="Seattle" maps:MapControl.Location="{x:Bind
SeattleLocation}"/>
    <TextBox Text="Bellevue" maps:MapControl.Location="{x:Bind
BellevueLocation}"/>
</maps:MapControl>

```

Set these locations by using a properties in your code-behind file.

C#

```

public Geopoint SeattleLocation { get; set; }
public Geopoint BellevueLocation { get; set; }

```

This example shows how to display two XAML controls contained within a **MapItemsControl**. These controls appear on the map at the data bound locations.

XAML

```

<maps:MapControl>
    <maps:MapItemsControl>
        <TextBox Text="Seattle" maps:MapControl.Location="{x:Bind
SeattleLocation}"/>
        <TextBox Text="Bellevue" maps:MapControl.Location="{x:Bind
BellevueLocation}"/>
    </maps:MapItemsControl>
</maps:MapControl>

```

This example displays a collection of XAML elements bound to a [MapItemsControl](#).

XAML

```

<maps:MapControl x:Name="MapControl" MapTapped="MapTapped"
MapDoubleTapped="MapTapped" MapHolding="MapTapped">
    <maps:MapItemsControl ItemsSource="{x:Bind LandmarkOverlays}">
        <maps:MapItemsControl.ItemTemplate>
            <DataTemplate>
                <StackPanel Background="Black" Tapped ="Overlay_Tapped">
                    <TextBlock maps:MapControl.Location="{Binding Location}"
Text="{Binding Title}"
                    maps:MapControl.NormalizedAnchorPoint="0.5,0.5"
FontSize="20" Margin="5"/>
                </StackPanel>
            </DataTemplate>
        </maps:MapItemsControl.ItemTemplate>
    </maps:MapItemsControl>
</maps:MapControl>

```

The `ItemsSource` property in the example above is bound to a property of type [IList](#) in the code-behind file.

C#

```

public sealed partial class Scenario1 : Page
{
    public IList LandmarkOverlays { get; set; }

    public MyClassConstructor()
    {
        SetLandMarkLocations();
        this.InitializeComponent();
    }

    private void SetLandMarkLocations()
    {
        LandmarkOverlays = new List<MapElement>();

        var pikePlaceIcon = new MapIcon
        {

```

```

        Location = new Geopoint(new BasicGeoposition
        { Latitude = 47.610, Longitude = -122.342 }),
        Title = "Pike Place Market"
    };

    LandmarkOverlays.Add(pikePlaceIcon);

    var SeattleSpaceNeedleIcon = new MapIcon
    {
        Location = new Geopoint(new BasicGeoposition
        { Latitude = 47.6205, Longitude = -122.3493 }),
        Title = "Seattle Space Needle"
    };

    LandmarkOverlays.Add(SeattleSpaceNeedleIcon);
}
}

```

Working with layers

The examples in this guide add elements to a [MapElementsLayer](#) collection. Then they show how to add that collection to the **Layers** property of the map control. In previous releases, this guide showed you how to add map elements to the [MapElements](#) collection as follows:

C#

```

var pikePlaceIcon = new MapIcon
{
    Location = new Geopoint(new BasicGeoposition
    { Latitude = 47.610, Longitude = -122.342 }),
    NormalizedAnchorPoint = new Point(0.5, 1.0),
    ZIndex = 0,
    Title = "Pike Place Market"
};

myMap.MapElements.Add(pikePlaceIcon);

```

While you can still use this approach, you'll miss out on some of the advantages of the new map layer model. By grouping your elements into layers, you can manipulate each layer independently from one another. For example, each layer has its own set of events so you can respond to an event on a specific layer and perform an action specific to that event.

Also, you can bind XAML directly to a [MapLayer](#). This is something that you can't do by using the [MapElements](#) collection.

One way you could do this is by using a view model class, code-behind a XAML page, and a XAML page.

View model class

C#

```
public class LandmarksViewModel
{
    public ObservableCollection<MapLayer> LandmarkLayer
    { get; } = new ObservableCollection<MapLayer>();

    public LandmarksViewModel()
    {
        var MyElements = new List<MapElement>();

        var pikePlaceIcon = new MapIcon
        {
            Location = new Geopoint(new BasicGeoposition
            { Latitude = 47.610, Longitude = -122.342 }),
            Title = "Pike Place Market"
        };

        MyElements.Add(pikePlaceIcon);

        var LandmarksLayer = new MapElementsLayer
        {
            ZIndex = 1,
            MapElements = MyElements
        };

        LandmarkLayer.Add(LandmarksLayer);
    }
}
```

Code-behind a XAML page

Connect the view model class to your code behind page.

C#

```
public LandmarksViewModel ViewModel { get; set; }

public myMapPage()
{
    this.InitializeComponent();
    this.ViewModel = new LandmarksViewModel();
}
```

XAML page

In your XAML page, bind to the property in your view model class that returns the layer.

XAML

```
<maps:MapControl
    x:Name="myMap" TransitFeaturesVisible="False" Loaded="MyMap_Loaded"
    Grid.Row="2"
    MapServiceToken="Your token" Layers="{x:Bind ViewModel.LandmarkLayer}"/>
```

Related topics

- [Bing Maps Developer Center](#) 
- [UWP map sample](#) 
- [Design guidelines for maps](#)
- [Build 2015 video: Leveraging Maps and Location Across Phone, Tablet, and PC in Your Windows Apps](#) 
- [UWP traffic app sample](#) 
- [MapIcon](#)
- [MapPolygon](#)
- [MapPolyline](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Display routes and directions on a map

Article • 06/21/2024

❗ Important

Bing Maps for Enterprise service retirement

The UWP [MapControl](#) and map services from the [Windows.Services.Maps](#) namespace rely on Bing Maps. Bing Maps for Enterprise is deprecated and will be retired, at which point the MapControl and services will no longer receive data.

For more information, see the [Bing Maps Developer Center](#) [↗] and [Bing Maps documentation](#).

❗ Note

[MapControl](#) and map services require a maps authentication key called a [MapServiceToken](#). For more info about getting and setting a maps authentication key, see [Request a maps authentication key](#).

Request routes and directions, and display them in your app.

❗ Note

If mapping isn't a core feature of your app, consider launching the Windows Maps app instead. You can use the `bingmaps:`, `ms-drive-to:`, and `ms-walk-to:` URI schemes to launch the Windows Maps app to specific maps and turn-by-turn directions. For more info, see [Launch the Windows Maps app](#).

An intro to MapRouteFinder results

Here's how the classes for routes and directions are related:

- The [MapRouteFinder](#) class has methods that get routes and directions. These methods return a [MapRouteFinderResult](#).
- The [MapRouteFinderResult](#) contains a [MapRoute](#) object. Access this object through the [Route](#) property of the [MapRouteFinderResult](#).

- The **MapRoute** contains a collection of **MapRouteLeg** objects. Access this collection through the **Legs** property of the **MapRoute**.
- Each **MapRouteLeg** contains a collection of **MapRouteManeuver** objects. Access this collection through the **Maneuvers** property of the **MapRouteLeg**.

Get a driving or walking route and directions by calling the methods of the **MapRouteFinder** class. For example, **GetDrivingRouteAsync** or **GetWalkingRouteAsync**.

When you request a route, you can specify the following things:

- You can provide a start point and end point only, or you can provide a series of waypoints to compute the route.

Stop waypoints adds additional route legs, each with their own Itinerary. To specify *stop* waypoints, use any of the **GetDrivingRouteFromWaypointsAsync** overloads.

Via waypoint defines intermediate locations between *stop* waypoints. They do not add route legs. They are merely waypoints that a route must pass through. To specify *via* waypoints, use any of the **GetDrivingRouteFromEnhancedWaypointsAsync** overloads.

- You can specify optimizations (For example: minimize the distance).
- You can specify restrictions (For example: avoid highways).

Display directions

The **MapRouteFinderResult** object contains a **MapRoute** object that you can access through its **Route** property.

The computed **MapRoute** has properties that provide the time to traverse the route, the length of the route, and the collection of **MapRouteLeg** objects that contain the legs of the route. Each **MapRouteLeg** object contains a collection of **MapRouteManeuver** objects. The **MapRouteManeuver** object contains directions that you can access through its **InstructionText** property.

Important

You must specify a maps authentication key before you can use map services. For more info, see [Request a maps authentication key](#).

```

using System;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.Services.Maps;
using Windows.Devices.Geolocation;
...
private async void button_Click(object sender, RoutedEventArgs e)
{
    // Start at Microsoft in Redmond, Washington.
    BasicGeoposition startLocation = new BasicGeoposition()
{Latitude=47.643,Longitude=-122.131};

    // End at the city of Seattle, Washington.
    BasicGeoposition endLocation = new BasicGeoposition() {Latitude =
47.604,Longitude= -122.329};

    // Get the route between the points.
    MapRouteFinderResult routeResult =
        await MapRouteFinder.GetDrivingRouteAsync(
            new Geopoint(startLocation),
            new Geopoint(endLocation),
            MapRouteOptimization.Time,
            MapRouteRestrictions.None);

    if (routeResult.Status == MapRouteFinderStatus.Success)
    {
        System.Text.StringBuilder routeInfo = new System.Text.StringBuilder();

        // Display summary info about the route.
        routeInfo.Append("Total estimated time (minutes) = ");

        routeInfo.Append(routeResult.Route.EstimatedDuration.TotalMinutes.ToString()
);
        routeInfo.Append("\nTotal length (kilometers) = ");
        routeInfo.Append((routeResult.Route.LengthInMeters /
1000).ToString());

        // Display the directions.
        routeInfo.Append("\n\nDIRECTIONS\n");

        foreach (MapRouteLeg leg in routeResult.Route.Legs)
        {
            foreach (MapRouteManeuver maneuver in leg.Maneuvers)
            {
                routeInfo.AppendLine(maneuver.InstructionText);
            }
        }

        // Load the text box.
        tbOutputText.Text = routeInfo.ToString();
    }
    else
    {
        tbOutputText.Text =

```

```

        "A problem occurred: " + routeResult.Status.ToString();
    }
}

```

This example displays the following results to the `tbOutputText` text box.

Output

```

Total estimated time (minutes) = 18.4833333333333
Total length (kilometers) = 21.847

DIRECTIONS
Head north on 157th Ave NE.
Turn left onto 159th Ave NE.
Turn left onto NE 40th St.
Turn left onto WA-520 W.
Enter the freeway WA-520 from the right.
Keep left onto I-5 S/Portland.
Keep right and leave the freeway at exit 165A towards James St..
Turn right onto James St.
You have reached your destination.

```

Display routes

To display a [MapRoute](#) on a [MapControl](#), construct a [MapRouteView](#) with the [MapRoute](#). Then, add the [MapRouteView](#) to the [Routes](#) collection of the [MapControl](#).

❗ Important

You must specify a maps authentication key before you can use map services or the map control. For more info, see [Request a maps authentication key](#).

C#

```

using System;
using Windows.Devices.Geolocation;
using Windows.Services.Maps;
using Windows.UI;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Controls.Maps;
...
private async void ShowRouteOnMap()
{
    // Start at Microsoft in Redmond, Washington.
    BasicGeoposition startLocation = new BasicGeoposition() { Latitude =
47.643, Longitude = -122.131 };

```

```

// End at the city of Seattle, Washington.
BasicGeoposition endLocation = new BasicGeoposition() { Latitude =
47.604, Longitude = -122.329 };

// Get the route between the points.
MapRouteFinderResult routeResult =
    await MapRouteFinder.GetDrivingRouteAsync(
        new Geopoint(startLocation),
        new Geopoint(endLocation),
        MapRouteOptimization.Time,
        MapRouteRestrictions.None);

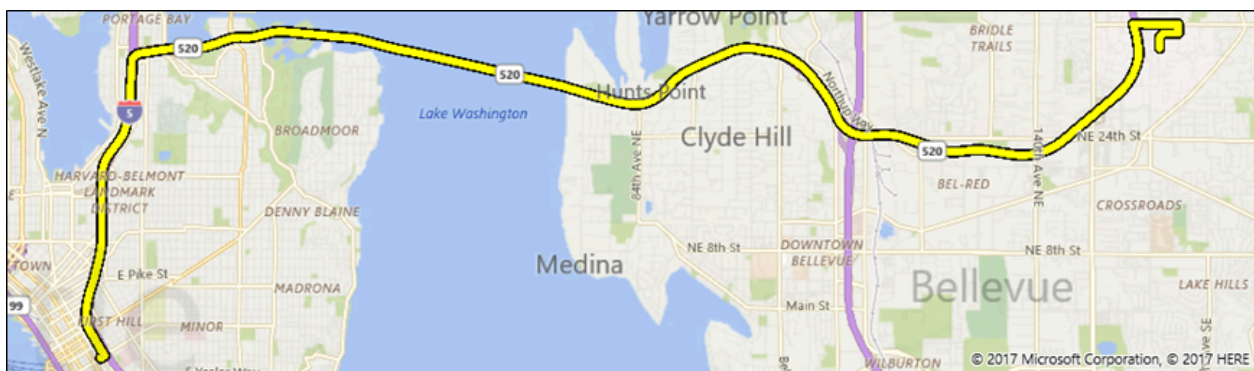
if (routeResult.Status == MapRouteFinderStatus.Success)
{
    // Use the route to initialize a MapRouteView.
    MapRouteView viewOfRoute = new MapRouteView(routeResult.Route);
    viewOfRoute.RouteColor = Colors.Yellow;
    viewOfRoute.OutlineColor = Colors.Black;

    // Add the new MapRouteView to the Routes collection
    // of the MapControl.
    MapWithRoute.Routes.Add(viewOfRoute);

    // Fit the MapControl to the route.
    await MapWithRoute.TrySetViewBoundsAsync(
        routeResult.Route.BoundingBox,
        null,
        Windows.UI.Xaml.Controls.Maps.MapAnimationKind.None);
}
}

```

This example displays the following on a [MapControl](#) named **MapWithRoute**.



Here's a version of this example that uses a *via* waypoint in between two *stop* waypoints:

```

C#

using System;
using Windows.Devices.Geolocation;
using Windows.Services.Maps;
using Windows.UI;

```

```

using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Controls.Maps;
...
private async void ShowRouteOnMap()
{
    Geolocator locator = new Geolocator();
    locator.DesiredAccuracyInMeters = 1;
    locator.PositionChanged += Locator_PositionChanged;

    BasicGeoposition point1 = new BasicGeoposition() { Latitude = 47.649693,
Longitude = -122.144908 };
    BasicGeoposition point2 = new BasicGeoposition() { Latitude = 47.6205,
Longitude = -122.3493 };
    BasicGeoposition point3 = new BasicGeoposition() { Latitude = 48.649693,
Longitude = -122.144908 };

    // Get Driving Route from point A to point B thru point C
    var path = new List<EnhancedWaypoint>();

    path.Add(new EnhancedWaypoint(new Geopoint(point1), WaypointKind.Stop));
    path.Add(new EnhancedWaypoint(new Geopoint(point2), WaypointKind.Via));
    path.Add(new EnhancedWaypoint(new Geopoint(point3), WaypointKind.Stop));

    MapRouteFinderResult routeResult = await
MapRouteFinder.GetDrivingRouteFromEnhancedWaypointsAsync(path);

    if (routeResult.Status == MapRouteFinderStatus.Success)
    {
        MapRouteView viewOfRoute = new MapRouteView(routeResult.Route);
        viewOfRoute.RouteColor = Colors.Yellow;
        viewOfRoute.OutlineColor = Colors.Black;

        myMap.Routes.Add(viewOfRoute);

        await myMap.TrySetViewBoundsAsync(
            routeResult.Route.BoundingBox,
            null,
            Windows.UI.Xaml.Controls.Maps.MapAnimationKind.None);
    }
}

```

Related topics

- [Bing Maps Developer Center](#) 
- [UWP map sample](#) 
- [Design guidelines for maps](#)

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Overlay tiled images on a map

Article • 06/21/2024

❗ Important

Bing Maps for Enterprise service retirement

The UWP [MapControl](#) and map services from the [Windows.Services.Maps](#) namespace rely on Bing Maps. Bing Maps for Enterprise is deprecated and will be retired, at which point the MapControl and services will no longer receive data.

For more information, see the [Bing Maps Developer Center](#) [↗] and [Bing Maps documentation](#).

ⓘ Note

[MapControl](#) and map services require a maps authentication key called a [MapServiceToken](#). For more info about getting and setting a maps authentication key, see [Request a maps authentication key](#).

Overlay third-party or custom tiled images on a map by using tile sources. Use tile sources to overlay specialized information such as weather data, population data, or seismic data; or use tile sources to replace the default map entirely.

Tiled image overview

Map services like Bing Maps cut maps into square tiles for quick retrieval and display. These tiles are 256 pixels by 256 pixels in size, and are pre-rendered at multiple levels of detail. Many third-party services also provide map-based data that's cut into tiles. Use tile sources to retrieve third-party tiles, or to create your own custom tiles, and overlay them on the map displayed in the [MapControl](#).

When you use tile sources, you don't have to write code to request or to position individual tiles. The [MapControl](#) requests tiles as it needs them. Each request specifies the X and Y coordinates and the zoom level for the individual tile. You simply specify the format of the Uri or filename to use to retrieve the tiles in the [UriFormatString](#) property. That is, you insert replaceable parameters in the base Uri or filename to indicate where to pass the X and Y coordinates and the zoom level for each tile.

Here's an example of the [UriFormatString](#) property for an [HttpMapTileDataSource](#) that shows the replaceable parameters for the X and Y coordinates and the zoom level.

syntax

```
http://www.<web service name>.com/z={zoomlevel}&x={x}&y={y}
```

(The X and Y coordinates represent the location of the individual tile within the map of the world at the specified level of detail. The tile numbering system starts from {0, 0} in the upper left corner of the map. For example, the tile at {1, 2} is in the second column of the third row of the grid of tiles.)

For more info about the tile system used by mapping services, see [Bing Maps Tile System](#).

Overlay tiles from a tile source

Overlay tiled images from a tile source on a map by using the [MapTileDataSource](#).

1. Instantiate one of the three tile data source classes that inherit from [MapTileDataSource](#).

- [HttpMapTileDataSource](#)
- [LocalMapTileDataSource](#)
- [CustomMapTileDataSource](#)

Configure the [UriFormatString](#) to use to request the tiles by inserting replaceable parameters in the base Uri or filename.

The following example instantiates an [HttpMapTileDataSource](#). This example specifies the value of the [UriFormatString](#) in the constructor of the [HttpMapTileDataSource](#).

C#

```
HttpMapTileDataSource dataSource = new HttpMapTileDataSource(  
    "http://www.<web service name>.com/z={zoomlevel}&x={x}&y={y}");
```

2. Instantiate and configure a [MapTileSource](#). Specify the [MapTileDataSource](#) that you configured in the previous step as the [DataSource](#) of the [MapTileSource](#).

The following example specifies the [DataSource](#) in the constructor of the [MapTileSource](#).

C#

```
MapTileSource tileSource = new MapTileSource(dataSource);
```

You can restrict the conditions in which the tiles are displayed by using properties of the [MapTileSource](#).

- Display tiles only within a specific geographic area by providing a value for the [Bounds](#) property.
- Display tiles only at certain levels of detail by providing a value for the [ZoomLevelRange](#) property.

Optionally, configure other properties of the [MapTileSource](#) that affect the loading or the display of the tiles, such as [Layer](#), [AllowOverstretch](#), [IsRetryEnabled](#), and [IsTransparencyEnabled](#).

3. Add the [MapTileSource](#) to the [TileSources](#) collection of the [MapControl](#).

C#

```
MapControl1.TileSources.Add(tileSource);
```

Overlay tiles from a web service

Overlay tiled images retrieved from a web service by using the [HttpMapTileDataSource](#).

1. Instantiate an [HttpMapTileDataSource](#).
2. Specify the format of the Uri that the web service expects as the value of the [UriFormatString](#) property. To create this value, insert replaceable parameters in the base Uri. For example, in the following code sample, the value of the [UriFormatString](#) is:

syntax

```
http://www.<web service name>.com/z={zoomlevel}&x={x}&y={y}
```

The web service has to support a Uri that contains the replaceable parameters {x}, {y}, and {zoomlevel}. Most web services (for example, Nokia, Bing, and Google) support Uris in this format. If the web service requires additional arguments that aren't available with the [UriFormatString](#) property, then you have to create a custom Uri. Create and return a custom Uri by handling the [UriRequested](#) event. For more info, see the [Provide a custom URI](#) section later in this topic.

3. Then, follow the remaining steps described previously in the [Tiled image overview](#).

The following example overlays tiles from a fictitious web service on a map of North America. The value of the [UriFormatString](#) is specified in the constructor of the [HttpMapTileDataSource](#). In this example, tiles are only displayed within the geographic boundaries specified by the optional [Bounds](#) property.

C#

```
private void AddHttpMapTileSource()
{
    // Create the bounding box in which the tiles are displayed.
    // This example represents North America.
    BasicGeoposition northWestCorner =
        new BasicGeoposition() { Latitude = 48.38544, Longitude =
-124.667360 };
    BasicGeoposition southEastCorner =
        new BasicGeoposition() { Latitude = 25.26954, Longitude = -80.30182
};
    GeoboundingBox boundingBox = new GeoboundingBox(northWestCorner,
southEastCorner);

    // Create an HTTP data source.
    // This example retrieves tiles from a fictitious web service.
    HttpMapTileDataSource dataSource = new HttpMapTileDataSource(
        "http://www.<web service name>.com/z={zoomlevel}&x={x}&y={y}");

    // Optionally, add custom HTTP headers if the web service requires them.
    dataSource.AdditionalRequestHeaders.Add("header name", "header value");

    // Create a tile source and add it to the Map control.
    MapTileSource tileSource = new MapTileSource(dataSource);
    tileSource.Bounds = boundingBox;
    MapControl1.TileSources.Add(tileSource);
}
```

C++/WinRT

```
...
#include <winrt/Windows.Devices.Geolocation.h>
#include <winrt/Windows.UI.Xaml.Controls.Maps.h>
...
void MainPage::AddHttpMapTileSource()
{
    Windows::Devices::Geolocation::BasicGeoposition northWest{ 48.38544,
-124.667360 };
    Windows::Devices::Geolocation::BasicGeoposition southEast{ 25.26954,
-80.30182 };
    Windows::Devices::Geolocation::GeoboundingBox boundingBox{ northWest,
southEast };
}
```

```

Windows::UI::Xaml::Controls::Maps::HttpMapTileDataSource dataSource{
    L"http://www.<web service name>.com/z={zoomlevel}&x={x}&y={y}" };

dataSource.AdditionalRequestHeaders().Insert(L"header name", L"header
value");

Windows::UI::Xaml::Controls::Maps::MapTileSource tileSource{ dataSource
};
tileSource.Bounds(boundingBox);

MapControl1().TileSources().Append(tileSource);
}
...

```

C++

```

void MainPage::AddHttpMapTileSource()
{
    BasicGeoposition northWest = { 48.38544, -124.667360 };
    BasicGeoposition southEast = { 25.26954, -80.30182 };
    GeoboundingBox^ boundingBox = ref new GeoboundingBox(northWest,
southEast);

    auto dataSource = ref new
Windows::UI::Xaml::Controls::Maps::HttpMapTileDataSource(
    "http://www.<web service name>.com/z={zoomlevel}&x={x}&y={y}");

    dataSource->AdditionalRequestHeaders->Insert("header name", "header
value");

    auto tileSource = ref new
Windows::UI::Xaml::Controls::Maps::MapTileSource(dataSource);
    tileSource->Bounds = boundingBox;

    this->MapControl1->TileSources->Append(tileSource);
}

```

Overlay tiles from local storage

Overlay tiled images stored as files in local storage by using the [LocalMapTileDataSource](#). Typically, you package and distribute these files with your app.

1. Instantiate a [LocalMapTileDataSource](#).
2. Specify the format of the file names as the value of the [UriFormatString](#) property. To create this value, insert replaceable parameters in the base filename. For example, in the following code sample, the value of the [UriFormatString](#) is:

syntax

```
Tile_{zoomlevel}_{x}_{y}.png
```

If the format of the file names requires additional arguments that aren't available with the [UriFormatString](#) property, then you have to create a custom Uri. Create and return a custom Uri by handling the [UriRequested](#) event. For more info, see the [Provide a custom URI](#) section later in this topic.

3. Then, follow the remaining steps described previously in the [Tiled image overview](#).

You can use the following protocols and locations to load tiles from local storage:

[Expand table](#)

Uri	More info
ms-appx:///	Points to the root of the app's installation folder. This is the location referenced by the Package.InstalledLocation property.
ms-appdata:///local	Points to the root of the app's local storage. This is the location referenced by the ApplicationData.LocalFolder property.
ms-appdata:///temp	Points to the app's temp folder. This is the location referenced by the ApplicationData.TemporaryFolder property.

The following example loads tiles that are stored as files in the app's installation folder by using the `ms-appx:///` protocol. The value for the [UriFormatString](#) is specified in the constructor of the [LocalMapTileDataSource](#). In this example, tiles are only displayed when the zoom level of the map is within the range specified by the optional [ZoomLevelRange](#) property.

C#

```
void AddLocalMapTileSource()
{
    // Specify the range of zoom levels
    // at which the overlaid tiles are displayed.
    MapZoomLevelRange range;
    range.Min = 11;
    range.Max = 20;

    // Create a local data source.
```

```

LocalMapTileDataSource dataSource = new LocalMapTileDataSource(
    "ms-appx:///TileSourceAssets/Tile_{zoomlevel}_{x}_{y}.png");

// Create a tile source and add it to the Map control.
MapTileSource tileSource = new MapTileSource(dataSource);
tileSource.ZoomLevelRange = range;
MapControl1.TileSources.Add(tileSource);
}

```

Provide a custom URI

If the replaceable parameters available with the [UriFormatString](#) property of the [HttpMapTileDataSource](#) or the [UriFormatString](#) property of the [LocalMapTileDataSource](#) aren't sufficient to retrieve your tiles, then you have to create a custom Uri. Create and return a custom Uri by providing a custom handler for the **UriRequested** event. The **UriRequested** event is raised for each individual tile.

1. In your custom handler for the **UriRequested** event, combine the required custom arguments with the [X](#), [Y](#), and [ZoomLevel](#) properties of the [MapTileUriRequestedEventArgs](#) to create the custom Uri.
2. Return the custom Uri in the [Uri](#) property of the [MapTileUriRequest](#), which is contained in the [Request](#) property of the [MapTileUriRequestedEventArgs](#).

The following example shows how to provide a custom Uri by creating a custom handler for the **UriRequested** event. It also shows how to implement the deferral pattern if you have to do something asynchronously to create the custom Uri.

```

C#

using Windows.UI.Xaml.Controls.Maps;
using System.Threading.Tasks;
...
var httpTileDataSource = new HttpMapTileDataSource();
// Attach a handler for the UriRequested event.
httpTileDataSource.UriRequested += HandleUriRequestAsync;
MapTileSource httpTileSource = new
MapTileSource(httpTileDataSource);
MapControl1.TileSources.Add(httpTileSource);
...
// Handle the UriRequested event.
private async void HandleUriRequestAsync(HttpMapTileDataSource
sender,
    MapTileUriRequestedEventArgs args)
{
    // Get a deferral to do something asynchronously.
    // Omit this line if you don't have to do something
asynchronously.
    var deferral = args.Request.GetDeferral();

```

```

        // Get the custom Uri.
        var uri = await GetCustomUriAsync(args.X, args.Y,
args.ZoomLevel);

        // Specify the Uri in the Uri property of the MapTileUriRequest.
        args.Request.Uri = uri;

        // Notify the app that the custom Uri is ready.
        // Omit this line also if you don't have to do something
asynchronously.
        deferral.Complete();
    }

    // Create the custom Uri.
    private async Task<Uri> GetCustomUriAsync(int x, int y, int
zoomLevel)
    {
        // Do something asynchronously to create and return the custom
Uri.
    }
}

```

Overlay tiles from a custom source

Overlay custom tiles by using the [CustomMapTileDataSource](#). Create tiles programmatically in memory on the fly, or write your own code to load existing tiles from another source.

To create or load custom tiles, provide a custom handler for the [BitmapRequested](#) event. The **BitmapRequested** event is raised for each individual tile.

1. In your custom handler for the [BitmapRequested](#) event, combine the required custom arguments with the [X](#), [Y](#), and [ZoomLevel](#) properties of the [MapTileBitmapRequestedEventArgs](#) to create or retrieve a custom tile.
2. Return the custom tile in the [PixelData](#) property of the [MapTileBitmapRequest](#), which is contained in the [Request](#) property of the [MapTileBitmapRequestedEventArgs](#). The [PixelData](#) property is of type [IRandomAccessStreamReference](#).

The following example shows how to provide custom tiles by creating a custom handler for the **BitmapRequested** event. This example creates identical red tiles that are partially opaque. The example ignores the [X](#), [Y](#), and [ZoomLevel](#) properties of the [MapTileBitmapRequestedEventArgs](#). Although this is not a real world example, the example demonstrates how you can create in-memory custom tiles on the fly. The example also shows how to implement the deferral pattern if you have to do something asynchronously to create the custom tiles.

C#

```
using Windows.UI.Xaml.Controls.Maps;
using Windows.Storage.Streams;
using System.Threading.Tasks;
...
    CustomMapTileDataSource customDataSource = new
CustomMapTileDataSource();
    // Attach a handler for the BitmapRequested event.
    customDataSource.BitmapRequested +=
customDataSource_BitmapRequestedAsync;
    customTileSource = new MapTileSource(customDataSource);
    MapControl1.TileSources.Add(customTileSource);
...
    // Handle the BitmapRequested event.
    private async void customDataSource_BitmapRequestedAsync(
        CustomMapTileDataSource sender,
        MapTileBitmapRequestedEventArgs args)
    {
        var deferral = args.Request.GetDeferral();
        args.Request.PixelData = await CreateBitmapAsStreamAsync();
        deferral.Complete();
    }

    // Create the custom tiles.
    // This example creates red tiles that are partially opaque.
    private async Task<RandomAccessStreamReference>
CreateBitmapAsStreamAsync()
    {
        int pixelHeight = 256;
        int pixelWidth = 256;
        int bpp = 4;

        byte[] bytes = new byte[pixelHeight * pixelWidth * bpp];

        for (int y = 0; y < pixelHeight; y++)
        {
            for (int x = 0; x < pixelWidth; x++)
            {
                int pixelIndex = y * pixelWidth + x;
                int byteIndex = pixelIndex * bpp;

                // Set the current pixel bytes.
                bytes[byteIndex] = 0xff;           // Red
                bytes[byteIndex + 1] = 0x00;       // Green
                bytes[byteIndex + 2] = 0x00;       // Blue
                bytes[byteIndex + 3] = 0x80;       // Alpha (0xff = fully
opaque)
            }
        }

        // Create RandomAccessStream from byte array.
        InMemoryRandomAccessStream randomAccessStream =
            new InMemoryRandomAccessStream();
```

```

        IOutputStream outputStream =
randomAccessStream.GetOutputStreamAt(0);
        DataWriter writer = new DataWriter(outputStream);
        writer.WriteBytes(bytes);
        await writer.StoreAsync();
        await writer.FlushAsync();
        return
RandomAccessStreamReference.CreateFromStream(randomAccessStream);
    }

```

C++/WinRT

```

...
#include <winrt/Windows.Storage.Streams.h>
...
Windows::Foundation::IAsyncOperation<Windows::Storage::Streams::InMemoryRandomAccessStream> MainPage::CustomRandomAccessStream()
{
    constexpr int pixelHeight{ 256 };
    constexpr int pixelWidth{ 256 };
    constexpr int bpp{ 4 };

    std::array<uint8_t, pixelHeight * pixelWidth * bpp> bytes;

    for (int y = 0; y < pixelHeight; y++)
    {
        for (int x = 0; x < pixelWidth; x++)
        {
            int pixelIndex{ y * pixelWidth + x };
            int byteIndex{ pixelIndex * bpp };

            // Set the current pixel bytes.
            bytes[byteIndex] = (byte)(std::rand() % 256);           // Red
            bytes[byteIndex + 1] = (byte)(std::rand() % 256);      // Green
            bytes[byteIndex + 2] = (byte)(std::rand() % 256);      // Blue
            bytes[byteIndex + 3] = (byte)((std::rand() % 56) + 200); //
Alpha (0xff = fully opaque)
        }
    }

    // Create RandomAccessStream from byte array.
    Windows::Storage::Streams::InMemoryRandomAccessStream
randomAccessStream;
    Windows::Storage::Streams::IOutputStream outputStream{
randomAccessStream.GetOutputStreamAt(0) };
    Windows::Storage::Streams::DataWriter writer{ outputStream };
    writer.WriteBytes(bytes);

    co_await writer.StoreAsync();
    co_await writer.FlushAsync();

    co_return randomAccessStream;
}

```

```
}  
...
```

C++

```
InMemoryRandomAccessStream^ TileSources::CustomRandomAccessStream::get()  
{  
    int pixelHeight = 256;  
    int pixelWidth = 256;  
    int bpp = 4;  
  
    Array<byte>^ bytes = ref new Array<byte>(pixelHeight * pixelWidth *  
bpp);  
  
    for (int y = 0; y < pixelHeight; y++)  
    {  
        for (int x = 0; x < pixelWidth; x++)  
        {  
            int pixelIndex = y * pixelWidth + x;  
            int byteIndex = pixelIndex * bpp;  
  
            // Set the current pixel bytes.  
            bytes[byteIndex] = (byte)(std::rand() % 256);           // Red  
            bytes[byteIndex + 1] = (byte)(std::rand() % 256);       // Green  
            bytes[byteIndex + 2] = (byte)(std::rand() % 256);       // Blue  
            bytes[byteIndex + 3] = (byte)((std::rand() % 56) + 200); //  
Alpha (0xff = fully opaque)  
        }  
    }  
  
    // Create RandomAccessStream from byte array.  
    InMemoryRandomAccessStream^ randomAccessStream = ref new  
InMemoryRandomAccessStream();  
    IOutputStream^ outputStream = randomAccessStream->GetOutputStreamAt(0);  
    DataWriter^ writer = ref new DataWriter(outputStream);  
    writer->WriteBytes(bytes);  
  
    create_task(writer->StoreAsync()).then([writer](unsigned int)  
    {  
        create_task(writer->FlushAsync());  
    }));  
  
    return randomAccessStream;  
}
```

Replace the default map

To replace the default map entirely with third-party or custom tiles: